

DEPARTMENT OF COMPUTER SCIENCE

Lab Manual

Data Mining

Prepared by

Qandeel Asim

Student ID: 2023-BS-AI-037

Bachelor of Artificial Intelligence

Submitted to

Mr. Arham

Table of Contents

(Right-click and select "Update Field" to refresh page numbers)

Lab 1:Introduction to Data Mining	3
Lab 2: CRISP-DM Framework Implementation	10
Lab 3: Data Cleaning	14
Lab 4: Normalization, Standardization & Discretization	18
Lab 5: Market Basket Analysis	26
Lab 6: Apriori Algorithm Implementation	30
Lab 7: FP-Growth Algorithm Analysis	37
Lab 8: Python Programming for Data Mining	40
Lab No 9: Decision Tree	47
Lab No 10: Naïve Bayes & K-Nearest Neighbors (KNN)	56
Lab No 11: Support Vector Machine (SVM) for Credit Card Fraud Detection	63
Lab No 12: Customer Segmentation	75
Lab No 13: Outlier Detection	84
Lab 14: Scraping & Sentiment Analysis – Product Review Analysis	95
Lab 15: Final Capstone – End-to-End Business Solution	102

Lab 1: Introduction to Data Mining

Data mining is the process of extracting useful patterns, relationships, and knowledge from large datasets. It combines techniques from statistics, machine learning, and database systems to analyze data and support decision-making. Data mining helps organizations discover hidden information, predict future trends, and improve business performance. It is widely used in various fields such as healthcare, finance, marketing, education, and e-commerce. The purpose of this experiment is to understand the basic concepts, techniques, and applications of data mining and to gain practical experience in analyzing data effectively.

● Introduction to KNIME Analytics Platform

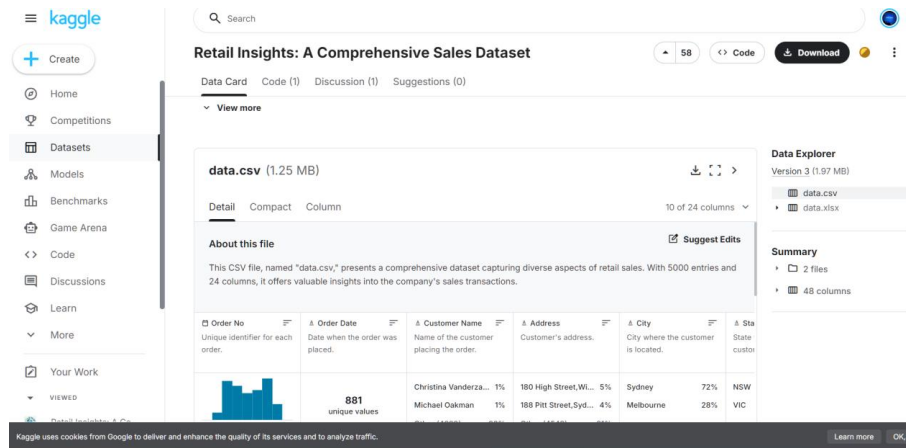
KNIME Analytics Platform is an open-source data analytics, reporting, and integration tool that enables users to perform data mining, machine learning, and data visualization through a graphical user interface. It allows users to create workflows by connecting different processing nodes without requiring extensive programming knowledge. KNIME supports data preprocessing, transformation, analysis, and predictive modeling, making it a powerful platform for data science and business intelligence applications. The objective of this experiment is to familiarize students with the KNIME environment, understand its basic components, and learn how to build and execute workflows for data analysis tasks.

Real World Problem

An electronics retailer has been collecting transactional data from its online and physical stores over the past year. The marketing department needs to analyze this data to understand overall sales performance, identify top-selling product categories, determine the average transaction value, and pinpoint the highest revenue-generating items. This lab demonstrates how to use KNIME Analytics Platform to perform these analyses through visual workflow construction.

The dataset consists of an Electronics Sales CSV file containing 50 transaction records. Each row represents a single purchase with the following fields: Transaction_ID, Product_Name, Category, Units_Sold, Unit_Price, Sale_Date, and Total_Sales.

Figure 1.0: Kaggle Dataset - Retail Insights



Practical Objectives

Upon completion of this laboratory exercise, students will be able to:

0. Import CSV datasets into KNIME workflows using the CSV Reader node.
1. Verify and correct column data types to ensure proper analysis.
2. Generate descriptive statistics to understand data distributions.
3. Create calculated fields using the Math Formula node.
4. Compute business metrics using GroupBy and Sorter nodes.

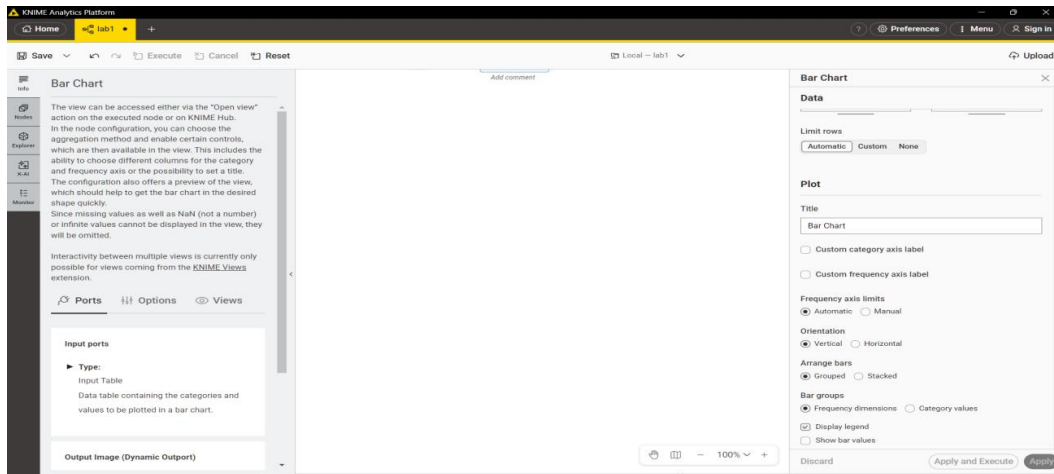
Procedure

Step 1: Create a New Workflow

Launch KNIME Analytics Platform. Navigate to File > New > New KNIME Workflow. Name the workflow "Electronics Sales Analysis" and click Finish. This creates a blank canvas where all nodes will be placed and connected.

Locate the CSV Reader node in the Node Repository panel on the left side. Drag it onto the workflow canvas. Double-click the node to open its configuration dialog. Click Browse and select the Electronics Sales CSV file. Click OK to confirm. Right-click the node and select Execute. Once execution completes, right-click again and choose View Table to verify the data has loaded correctly.

Figure 1.1: KNIME Analytics Platform - Bar Chart Configuration

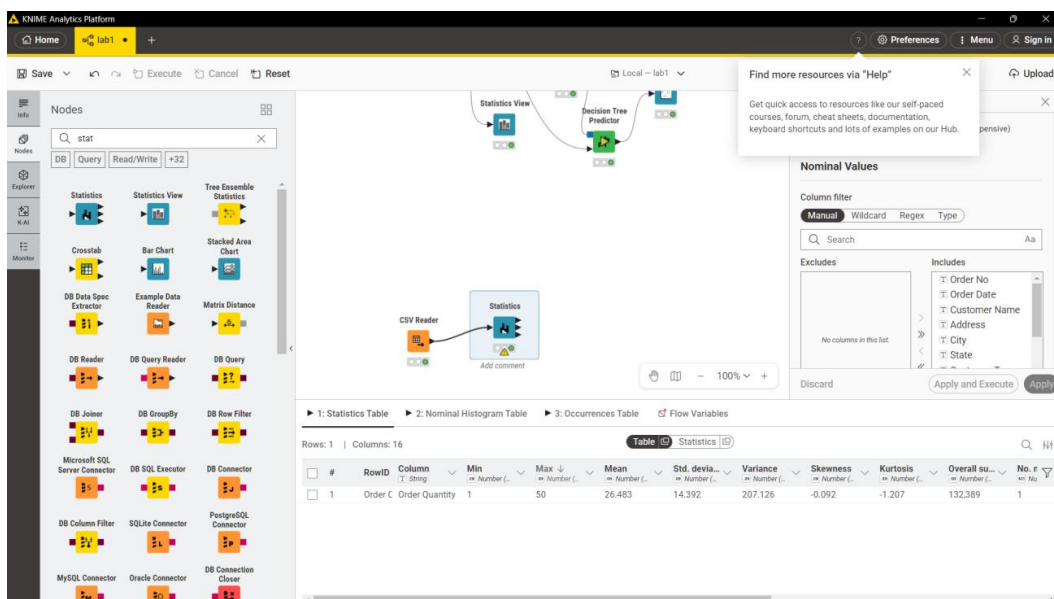


Step 3: Verify Data Types

After loading, inspect the column types displayed in the CSV Reader output. Units_Sold must be Integer, Unit_Price must be Double, Sale_Date must be Date type, and Category must be String. If any column is incorrectly typed (for example, if Unit_Price is read as String), apply the appropriate conversion node before proceeding. Use String to Number for numeric columns.

Add a Statistics node to the canvas and connect it to the output port of the CSV Reader. Execute the node and view the results. The output displays mean, minimum, maximum, standard deviation, and sum for each numeric column. This step provides an immediate overview of data distribution and helps identify any anomalies before proceeding with analysis.

Figure 1.3: Statistics Node and Data View



Rows: 5000 | Columns: 24

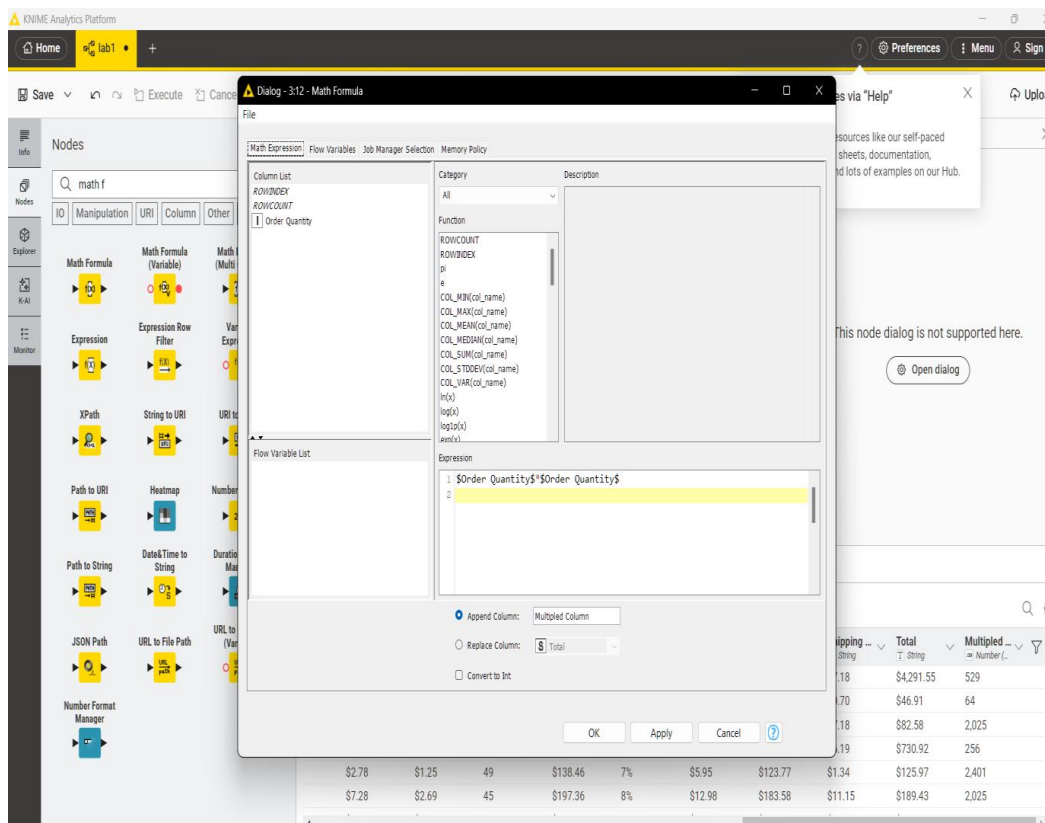
Product ...	Product C...	Product C...	Ship Mode	Ship Date	Cost Price	Retail Price	Profit Ma...	Order Qu...	Sub Total	Disco
T: String	T: String	T: String	T: String	T: String	T: String	T: String	T: String	Number (L...	T: String	T: Stri
Binder Posts	Office Supplies	Small Box	Regular Air	14-08-2015	\$3.50	\$5.74	\$2.24	31	\$213.34	0%
Artisan 479 Lab	Office Supplies	Wrap Bag	Regular Air	20-06-2016	\$1.59	\$2.61	\$1.02	23	\$66.54	5%
Laser DVD-RAM	Furniture	Small Pack	Regular Air	30-09-2016	\$20.18	\$35.41	\$15.23	19	\$929.40	9%
Smiths Colored	Office Supplies	Small Pack	Regular Air	18-12-2015	\$19.83	\$30.98	\$11.15	49	\$1,999.69	7%
Artisan Hi-Liter	Office Supplies	Small Pack	Regular Air	06-11-2016	\$2.98	\$5.84	\$2.86	35	\$115.40	2%
210 Trimline Ph	Office Supplies	Small Pack	Regular Air	15-05-2016	\$9.91	\$15.99	\$6.08	2	\$864.20	5%

Add a Math Formula node and connect it to the CSV Reader. In the node configuration, create a new column named `Total_Sales` using the formula:

`$Units_Sold$ * $Unit_Price$`

Execute the node. Each row in the output now contains the total sales value for that transaction, which serves as the basis for all metric calculations.

Figure 1.4: Math Formula Node Configuration



Add a GroupBy node with no grouping column selected. In the Manual Aggregation tab, select the `Total_Sales` column and set the aggregation method to Sum. Execute the node. The single output row shows the total revenue across all transactions.

Figure 1.5: GroupBy Configuration for Aggregation

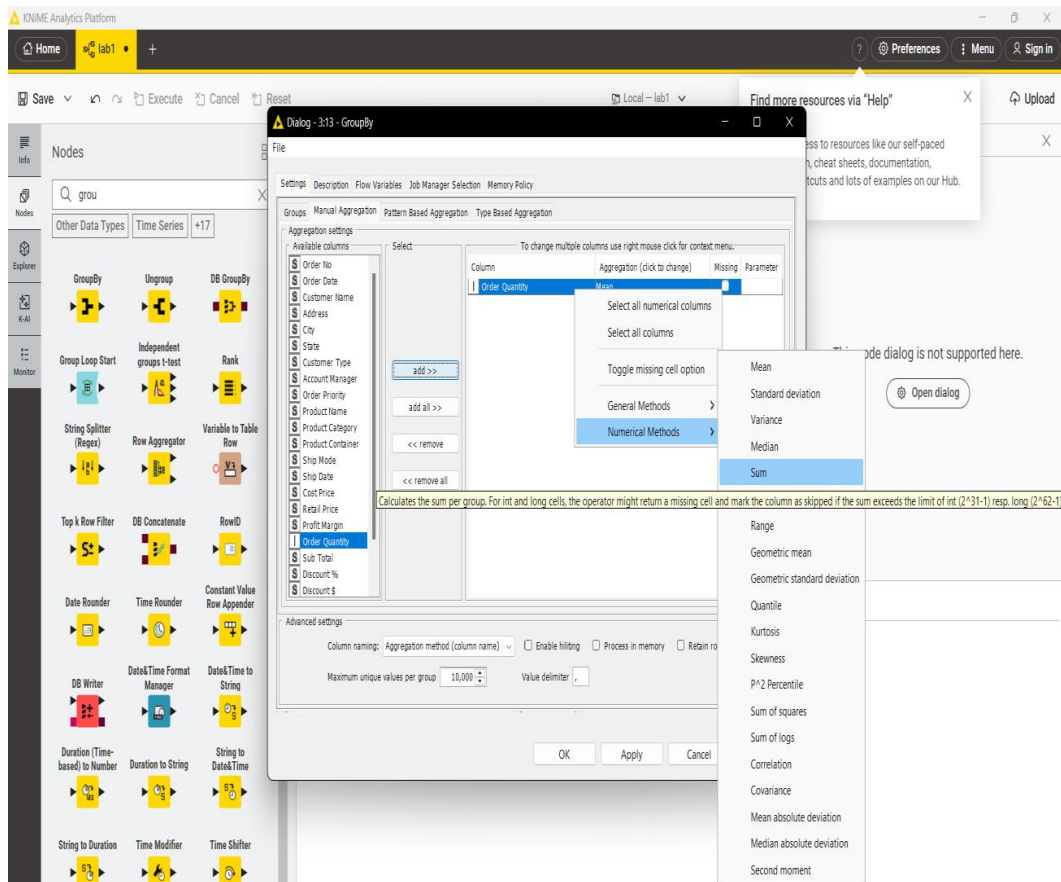
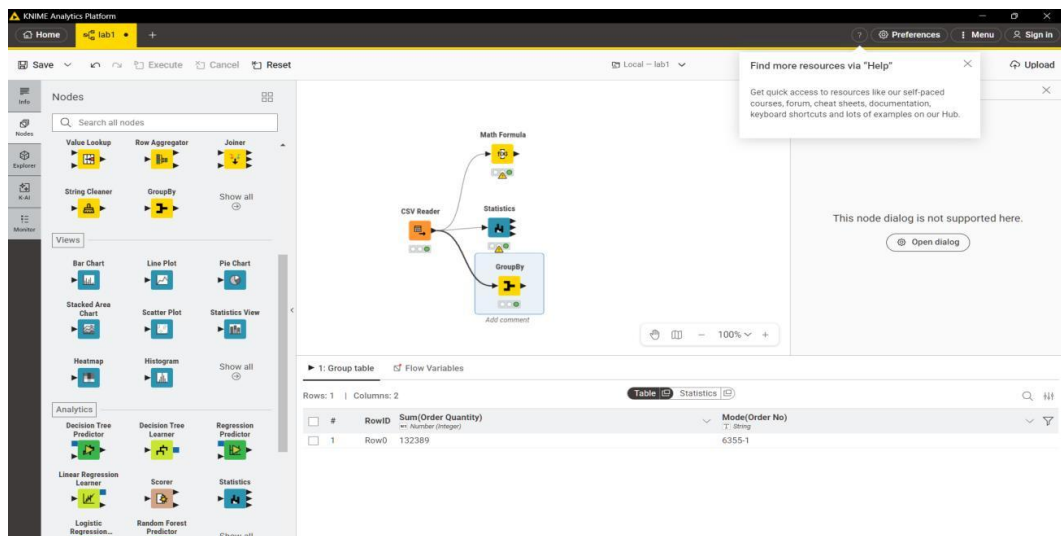


Figure 1.6: KPI Results - Total Revenue



Rows: 1 | Columns: 1

☐	#	RowID	Sum(Order Quantity)
			123 Number (Integer)
☐	1	Row0	136987

KPI 2: Average Transaction Value

Add a GroupBy node and group by Transaction_ID, aggregating Total_Sales using Sum. This produces the total value per transaction. Connect a second GroupBy node with no grouping column, aggregating the result using Mean. The output is the average transaction value.

Add a GroupBy node and group by the Category column, aggregating Total_Sales using Sum. Connect a Sorter node configured to sort by Total_Sales in descending order. The resulting table ranks each product category from highest to lowest earnings.

Figure 1.7: KPI Results - Category Revenue

The screenshot displays the KNIME Analytics Platform interface. On the left, a 'Nodes' panel lists various data processing tools. The main workspace shows a workflow: a 'CSV Reader' node connected to a 'Statistics' node, which is then connected to a 'GroupBy' node. The 'GroupBy' node is configured with 'Product Category' as the grouping column and 'Sum' as the aggregation function. The output of the 'GroupBy' node is connected to a 'Sorter' node. A 'Find more resources via "Help"' tooltip is visible over the 'Statistics' node. Below the workflow, the 'Table' view shows the results of the 'Sorter' node, which are sorted by 'Max(Profit Margin)' in descending order.

#	RowID	Product Category	Max(Profit Margin)
1	Row0	Furniture	\$97.47
2	Row1	Office Supplies	\$97.47
3	Row2	Technology	\$97.47

Add a GroupBy node and group by Product_Name, aggregating Total_Sales using Sum. Connect a Sorter node sorted by Total_Sales in descending order. The top rows of the output identify the highest revenue-generating products.

Figure 1.8: KPI Results - Top Products by Revenue

The screenshot shows the KNIME Analytics Platform interface. The workflow consists of the following nodes: CSV Reader, Statistics, GroupBy, and Sorter. A configuration dialog for the Sorter node is open, showing the following settings:

- Criterion 1: Column: Product Category, Order: Descending
- Buttons: Add sorting criterion, Show advanced settings, Discard, Apply and Execute, Apply

Below the workflow, a table displays the top 3 products by revenue:

#	RowID	Product Category	First(Product Name)
1	Row0	Furniture	12 Colored Short Pencils
2	Row1	Office Supplies	UGen Ultra Professional Cordless Optical Suite
3	Row2	Technology	Smiths Metal Binder Clips

Lab 2: CRISP-DM Framework Implementation

Real World Problem

A subscription-based software company is experiencing an increase in customer cancellations. The management team wants to predict which customers are at risk of churning so that proactive retention strategies can be implemented. This lab applies the CRISP-DM (Cross-Industry Standard Process for Data Mining) framework to structure the entire data mining process from business understanding through deployment.

Dataset

Customer Subscription Dataset containing 50 records with customer demographics, subscription details, usage patterns, support interactions, and a binary churn label indicating whether the customer discontinued their subscription.

Practical Objectives

By the end of this lab, students will be able to:

5. Apply all six CRISP-DM phases to a real-world churn prediction problem.
6. Document business objectives and success criteria.
7. Explore and prepare datasets for modeling.
8. Build and evaluate predictive models using Orange Data Mining.
9. Develop a deployment plan for the solution.

Procedure

Step 1: Business Understanding

Define the business goal: predict customer churn. Identify success criteria: a model achieving at least 82% accuracy and an ROC-AUC score above 0.78. Document what churn means in this context (customer cancels subscription within 30 days) and list available data sources.

Open Orange Data Mining. Load the Customer Subscription CSV using the File widget. Connect a Data Table widget to inspect rows and columns. Use the Feature Statistics widget to view distributions, missing value counts, and class balance. Record the churn rate as a baseline metric.

Figure 2.1: Customer Churn Dataset Description

Customer churn refers to the phenomenon where customers discontinue their relationship or subscription with a company or service provider. It represents the rate at which customers stop using a company's products or services within a specific period. Churn is an important metric for businesses as it directly impacts revenue, growth, and customer retention.

In the context of the Churn dataset, the churn label indicates whether a customer has churned or not. A churned customer is one who has decided to discontinue their subscription or usage of the company's services. On the other hand, a non-churned customer is one who continues to remain engaged and retains their relationship with the company.

Understanding customer churn is crucial for businesses to identify patterns, factors, and indicators that contribute to customer attrition. By analyzing churn behavior and its associated features, companies can develop strategies to retain existing customers, improve customer satisfaction, and reduce customer turnover. Predictive modeling techniques can also be applied to forecast and proactively address potential churn, enabling companies to take proactive measures to retain at-risk customers.

customer_churn_dataset-testing-master.csv (3.28 MB) ⬇️ ⌂ ➡️

Detail

Compact

Column

10 of 12 columns ▾

About this file

 **Suggest Edits**

The testing file for the churn dataset consists of 64,374 customer records and serves as a separate dataset for evaluating the performance and generalization capabilities of trained churn prediction models. Each record in the testing file corresponds to a customer and contains the same set of features as the training file, such as age, gender, tenure, usage frequency, support calls, payment delay, subscription type, contract length, total spend, and last interaction. However, the churn labels are not included in the testing file as they are used for assessing the accuracy and effectiveness of the churn prediction models. The testing file allows businesses to assess the predictive power of their trained models on unseen data and gain insights into how well the models generalize to new customers. By analyzing the model's performance on the testing file, businesses can gauge the effectiveness of their churn prediction strategies and make informed decisions to optimize customer retention efforts.

Use the Preprocess widget to handle missing values (impute with median for numeric, most frequent for categorical), encode categorical features using the Continuize widget, and normalize all numeric attributes. Remove any identifier columns such as Customer_ID that carry no predictive value.

Figure 2.2: Data Table and Feature Statistics in Orange

Data Table - Orange

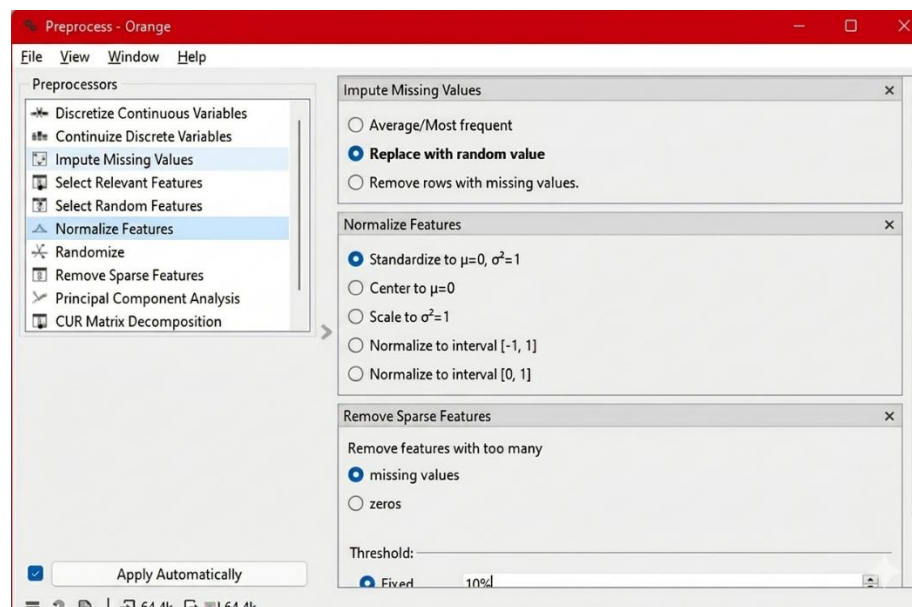
Info
 64374 instances (no missing data)
 12 features
 No target variable.
 No meta attributes.

Variables
☒ Show variable labels (if present)
☐ Visualize numeric values
☒ Color by instance classes

Selection
 Clear Selection
☒ Select full rows

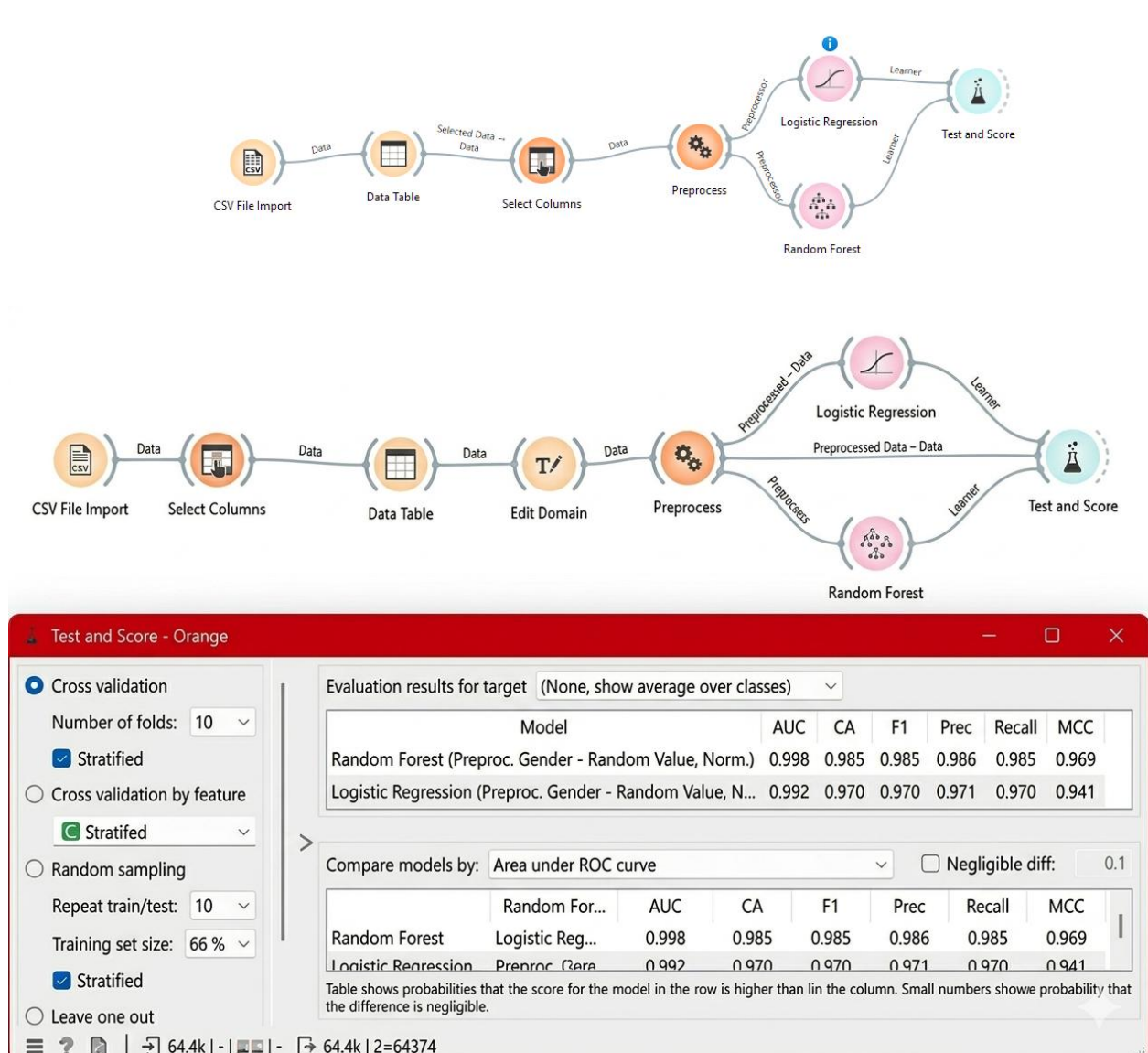
Restore Original Order
☒ Send Automatically

	CustomerID	Age	Gender	Tenure	Usage Frequency	Churned
1	1	30	Male	15	8	0
2	2	27	Male	3	2	0
3	3	47	Male	27	10	0
4	4	35	Male	9	12	0
5	5	29	Female	58	24	0
6	6	27	Female	41	14	0
7	7	47	Female	37	15	0
8	8	54	Female	36	11	0
9	9	36	Male	20	5	0
10	10	65	Male	8	4	0
11	11	46	Female	42	27	0
12	12	56	Male	13	23	0
13	13	31	Male	2	7	0
14	14	42	Male	46	27	0
16	15	59	Male	21	17	0
16	16	35	Female	1	3	0
17	17	29	Female	54	35	0
18	18	45	Male	9	30	0



Add a Logistic Regression learner and a Gradient Boosting learner to the canvas. Connect both to the preprocessed data. Link them to the Test and Score widget configured with 10-fold cross-validation and the Churned column set as the target variable.

Figure 2.3: Orange Data Mining Workflow for Churn Prediction



View the Test and Score results. Record Accuracy, Precision, Recall, F1, and ROC-AUC for both models. Connect the ROC Analysis widget to visualize the curves. Compare both models against the success criteria defined in Step 1 and select the better performer.

Figure 2.4: Model Evaluation Results

Evaluation results for target (None, show average over classes)						
Model	AUC	CA	F1	Prec	Recall	MCC
Random Forest (Preproc. Gender - Random Value, Norm.)	0.998	0.985	0.985	0.986	0.985	0.969
Logistic Regression (Preproc. Gender - Random Value, N...)	0.992	0.970	0.970	0.971	0.970	0.941

Lab 3: Data Cleaning

Data cleaning is the process of identifying and correcting errors, missing values, duplicate records, and inconsistent data in a dataset to improve its quality and accuracy. For example, a student dataset may contain missing ages, invalid salary values, duplicate entries, or incorrect phone numbers. During data cleaning, these issues are handled by removing duplicates, filling missing values, correcting invalid data, and standardizing the format of the dataset. This process is important because clean data helps in accurate analysis, better decision-making, and improved machine learning model performance.

Real World Problem

A medical research institute is preparing patient health records collected from multiple clinics for a cardiovascular risk assessment study. The raw data contains missing diagnostic values, duplicate patient entries from database migrations, and noisy measurements from monitoring devices. Before any analysis can proceed, the data must be cleaned to a standard suitable for clinical machine learning applications.

Dataset

Healthcare Dataset containing 30 patient records including age, gender, heart rate, cholesterol levels, BMI, diagnosis categories, and admission dates with intentionally introduced missing values, duplicates, and outliers for educational purposes.

Practical Objectives

Students will be able to:

10. Identify and handle missing values using multiple imputation strategies.
11. Detect and remove duplicate records based on unique identifiers.
12. Apply noise filtering using smoothing techniques.
13. Calculate data quality scores before and after cleaning.

Procedure

Use the Read CSV operator to import the healthcare dataset. Connect a Retrieve operator to the process and run it. View the data in the Results panel. Note the total row count (30), column types, and any columns flagged with missing indicators.

Figure 3.1: Loading and Profiling Healthcare Data


```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import warnings
warnings.filterwarnings('ignore')

df = pd.read_csv('healthcare_dataset.csv')

print('Dataset Shape:', df_raw.shape)
print('\nFirst 5 rows:')
df_raw.head()
```

Dataset Shape: (100, 7)

First 5 rows:

	PatientID	Age	Gender	BloodPressure	Glucose	BMI	Diagnosis
0	1	58	Female	116.785133	86.101378	20.288436	Healthy
1	2	71	Female	109.662087	96.806629	10.179522	Healthy
2	3	48	None	NaN	60.986073	32.617709	Healthy
3	4	34	Female	110.713960	150.008702	37.716026	Healthy
4	5	62	Female	125.342397	69.996707	25.287056	Healthy

Next steps: [Generate code with df_raw](#) [New interactive sheet](#)

Add the Replace Missing Values operator. For numeric columns such as Age and Cholesterol, set the replacement method to Average. For categorical columns such as Gender, set it to Most Frequent Value. For columns where more than 35% of values are missing, use the Remove Useless Attributes operator to drop them entirely.

Figure 3.2: Handling Missing Values

```
Step 1: Load and Profile the Data

# Profile the dataset
print('=== DATASET PROFILE ===')
print(f'Total Rows : {df_raw.shape[0]}')
print(f'Total Columns : {df_raw.shape[1]}')
print(f'\nData Types:')
print(df_raw.dtypes)
print(f'\nMissing Values per Column:')
print(df_raw.isnull().sum())
print(f'\nTotal Missing Cells : {df_raw.isnull().sum().sum()}')
print(f'Total Duplicate Rows: {df_raw.duplicated(subset="PatientID").sum()}')
```

=== DATASET PROFILE ===

Total Rows : 100
Total Columns : 7

Data Types:

PatientID	int64
Age	int64
Gender	object
BloodPressure	float64
Glucose	float64
BMI	float64
Diagnosis	object
dtype:	object

Missing Values per Column:

PatientID	0
Age	0
Gender	9
BloodPressure	5
Glucose	10
BMI	8
Diagnosis	7
dtype:	int64

Total Missing Cells : 39
Total Duplicate Rows: 10

Step 2: Handle Missing Values

```
df_clean = df_raw.copy()

# --- Numeric columns: fill with mean ---
numeric_cols = ['Glucose', 'BMI', 'BloodPressure']
for col in numeric_cols:
    before = df_clean[col].isnull().sum()
    df_clean[col].fillna(df_clean[col].mean(), inplace=True)
    after = df_clean[col].isnull().sum()
    print(f'{col}: {before} missing → filled with mean ({df_raw[col].mean():.2f}) → {after} missing')

# --- Categorical columns: fill with most frequent value ---
cat_cols = ['Gender', 'Diagnosis']
for col in cat_cols:
    before = df_clean[col].isnull().sum()
    most_freq = df_clean[col].mode()[0]
    df_clean[col].fillna(most_freq, inplace=True)
    after = df_clean[col].isnull().sum()
    print(f'{col}: {before} missing → filled with "{most_freq}" → {after} missing')

print(f'\nTotal missing after handling: {df_clean.isnull().sum().sum()}')

Glucose: 10 missing → filled with mean (101.00) → 0 missing
BMI: 8 missing → filled with mean (26.49) → 0 missing
BloodPressure: 5 missing → filled with mean (118.62) → 0 missing
Gender: 9 missing → filled with "Male" → 0 missing
Diagnosis: 7 missing → filled with "Healthy" → 0 missing
Total missing after handling: 0
```

Add the Remove Duplicates operator. Configure it to detect duplicate rows based on Patient_ID. Keep only the first occurrence. Record how many rows were removed. Then switch to Python and validate the result using pandas: `df.duplicated().sum()` before and after.

Figure 3.3: Removing Duplicate Records

Step 3: Remove Duplicate Records

```
# Check duplicates BEFORE
dups_before = df_clean.duplicated(subset='PatientID').sum()
print(f'Duplicate rows BEFORE removal: {dups_before}')
print(f'Total rows BEFORE : {len(df_clean)}')

# Remove duplicates — keep first occurrence
df_clean = df_clean.drop_duplicates(subset='PatientID', keep='first')

dups_after = df_clean.duplicated(subset='PatientID').sum()
print(f'\nDuplicate rows AFTER removal : {dups_after}')
print(f'Total rows AFTER : {len(df_clean)}')
print(f'Rows removed : {dups_before}')

Duplicate rows BEFORE removal: 10
Total rows BEFORE : 100

Duplicate rows AFTER removal : 0
Total rows AFTER : 90
Rows removed : 10
```

Export the cleaned data from RapidMiner as CSV. Load it in Python using pandas. Apply a rolling mean smoothing with a window of 3 on the Heart_Rate column to reduce measurement noise. Use box plots (via matplotlib) to visualize outliers in Cholesterol and BMI before and after filtering. Cap extreme values at the 5th and 95th percentiles using `np.clip`.

Figure 3.4: Noise Filtering with Box Plot Visualization



Lab 4: Normalization, Standardization & Discretization

Real World Problem

A financial institution is preparing a credit application dataset for a default risk prediction model. Raw financial data contains attributes with vastly different scales (monthly income vs. credit score), skewed distributions, and continuous variables that need to be converted into meaningful categories for rule-based decision systems. Preprocessing must be applied before model training can begin.

Summary Statistics

Summary statistics provide a brief description of the main features of a dataset. These measures help in understanding the data distribution and identifying patterns or trends.

Common statistical measures include:

- **Mean:** The average value
- **Median:** The middle value
- **Mode:** The most frequently occurring value
- **Variance:** Measures how spread out the data is
- **Standard Deviation:** Indicates how much the values deviate from the mean

Example:

In a dataset of website traffic, summary statistics can help determine the average number of daily visitors and observe fluctuations over time.

1. Summary Statistics

```
: import pandas as pd

# Load the dataset
df = pd.read_csv('loan_prediction_dataset.csv')
```

```
: print("Summary Statistics:")
display(df.describe())
```

Summary Statistics:

	Age	Income	Credit_Score	Loan_Amount	Loan_Term	Loan_Approved
count	2000.000000	2000.000000	2000.000000	2000.000000	2000.000000	2000.000000
mean	43.805500	84533.585000	577.055000	25460.315000	35.47800	0.171000
std	14.929203	37771.169751	157.525951	14116.737774	16.98587	0.376603
min	18.000000	20155.000000	300.000000	1060.000000	12.00000	0.000000
25%	31.000000	50925.250000	440.000000	13444.250000	24.00000	0.000000
50%	44.000000	84073.500000	578.500000	25446.000000	36.00000	0.000000
75%	56.000000	117523.250000	715.250000	37949.250000	48.00000	0.000000
max	69.000000	149992.000000	849.000000	49994.000000	60.00000	1.000000

2. Normalization (Min-Max Scaling)

```
]]: from sklearn.preprocessing import MinMaxScaler

# Initialize the scaler
scaler_minmax = MinMaxScaler()

# We will normalize 'Income' and 'Loan_Amount'
df_normalized = df.copy()
cols_to_normalize = ['Income', 'Loan_Amount']
df_normalized[cols_to_normalize] = scaler_minmax.fit_transform(df[cols_to_normalize])

print("Normalized Data (First 5 rows):")
print(df_normalized[cols_to_normalize].head())
```

Normalized Data (First 5 rows):

	Income	Loan_Amount
0	0.474695	0.285323
1	0.637137	0.407753
2	0.297850	0.789369
3	0.824364	0.323415
4	0.042361	0.514898

3. Standardization (Z-score Scaling)

```
[5]: from sklearn.preprocessing import StandardScaler

# Initialize the scaler
scaler_std = StandardScaler()

# We will standardize 'Age' and 'Credit_Score'
df_standardized = df.copy()
cols_to_standardize = ['Age', 'Credit_Score']
df_standardized[cols_to_standardize] = scaler_std.fit_transform(df[cols_to_standardize])

print("Standardized Data (First 5 rows):")
print(df_standardized[cols_to_standardize].head())
```

Standardized Data (First 5 rows):

	Age	Credit_Score
0	0.817026	-1.543338
1	1.688020	1.294999
2	0.147031	1.282300
3	-0.790963	-1.352846
4	1.085024	-1.714781

4. Discretization (Binning)

```
[7]: df_discretized = df.copy()
df_discretized['Income_Category'] = pd.cut(df['Income'],
                                          bins=3,
                                          labels=['Low', 'Medium', 'High'])

from sklearn.preprocessing import KBinsDiscretizer
est = KBinsDiscretizer(n_bins=3, encode='ordinal', strategy='uniform')
df_discretized['Age_Bins'] = est.fit_transform(df[['Age']])

print("Discretized Data (First 5 rows):")
print(df_discretized[['Income', 'Income_Category', 'Age', 'Age_Bins']].head())
```

Discretized Data (First 5 rows):

	Income	Income_Category	Age	Age_Bins
0	81788	Medium	56	2.0
1	102879	Medium	69	2.0
2	58827	Low	46	1.0
3	127188	High	32	0.0
4	25655	Low	60	2.0

RAPID MINER :

RapidMiner is a powerful, user- friendly data science platform designed for data preparation, machine learning, and predictive analytics. It uses a visual workflow approach where tasks are represented as “operators” connected in a process, making it ideal for lab manuals and teaching

Dataset

Credit Application Dataset containing 20 applicant records with monthly income, loan amount requested, credit score, employment history, and a binary loan approval label.

Practical Objectives

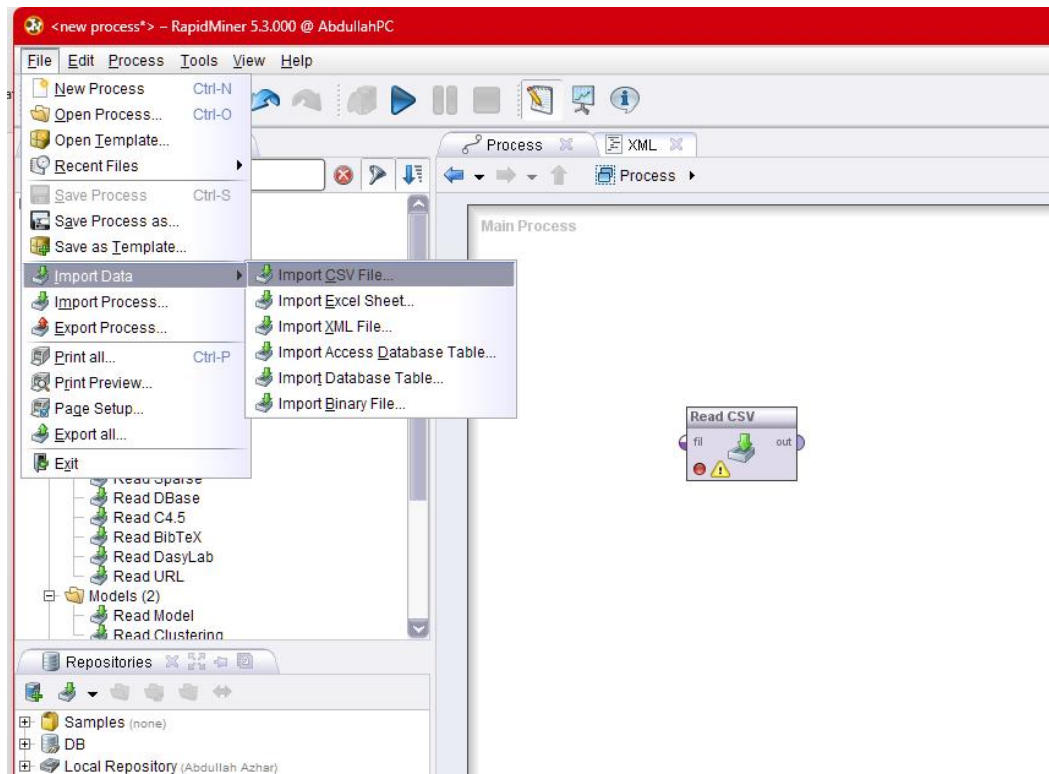
Students will:

14. Apply min-max normalization to scale numeric features.
15. Apply Z-score standardization for statistical analysis.
16. Perform equal-width discretization on continuous variables.
17. Observe how each transformation affects data distribution.

Procedure

In RapidMiner, import the Credit Application CSV using the Read CSV operator. Add a Statistics operator and execute the process to capture the mean, variance, minimum, and maximum for Monthly_Income and Loan_Amount. Save these baseline values for comparison after preprocessing.

Figure 4.1: Importing CSV File in RapidMiner



Add the Normalize operator. Set the method to Range Transformation with a minimum of 0 and maximum of 1. Apply it to Monthly_Income and Loan_Amount. Execute and view the transformed values. Verify that all values now fall within $[0, 1]$ and record the updated statistics.

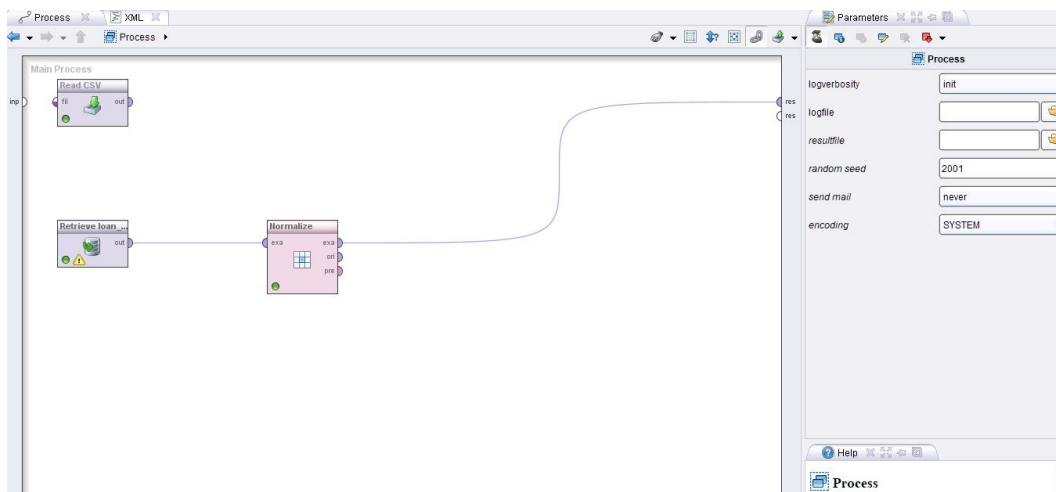
Figure 4.2: Min-Max Normalization Results

ExampleSet (Normalize)

☒ Data View ☐ Meta Data View ☐ Plot View ☐ A

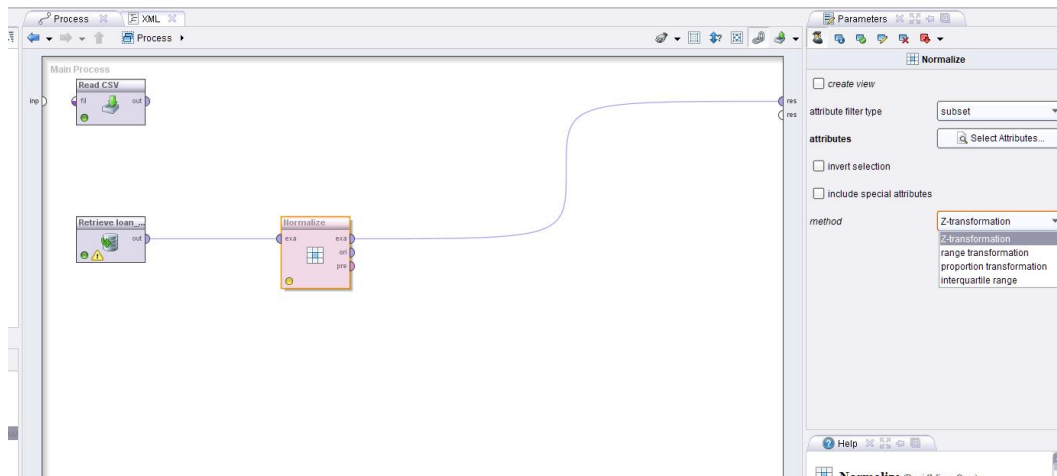
ExampleSet (20000 examples, 1 special attribute, 21 i

Row No.	loan_paid_...	annual_inc...	loan_amount
1	1	0.046	0.344
2	1	0.036	0.457
3	1	0.051	0.065
4	1	0.015	0.297
5	1	0.049	0.331
6	1	0.126	0.252
7	0	0.047	0.111
8	1	0.110	0.300



Replace the Normalize operator with a fresh one and set the method to Standardization (Z-transformation). Apply to the same columns. Execute and verify the resulting mean is approximately 0 and standard deviation approximately 1. Compare this output against the min-max result and note the practical differences.

Figure 4.3: Z-Score Standardization Process

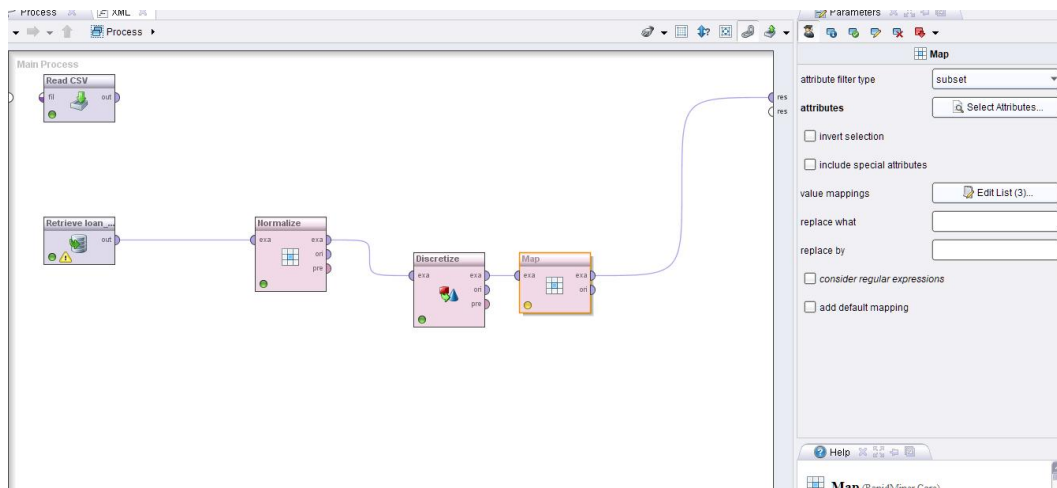


ExampleSet (20000 examples, 1 special attribute, 2 float)

Row No.	loan_paid_...	annual_inc...	loan_amount
1	1	-0.674	0.238
2	1	-0.815	0.876
3	1	-0.606	-1.336
4	1	-1.105	-0.022
5	1	-0.636	0.165
6	1	0.419	-0.280
7	0	-0.659	-1.072
8	1	0.316	0.001
9	1	0.676	-1.700

Add the Discretize operator. Set the method to Equal Width with 3 bins for Monthly_Income. Label the bins as Low, Medium, and High. Execute and view the new categorical column. Observe how the continuous income variable has been transformed into an ordinal attribute suitable for rule-based classifiers.

Figure 4.4: Discretization Process in RapidMiner



sampleSet (20000 examples, 1 special attribute, 21 reg

row No.	loan_paid_...	annual_inc...	loan_amount
	1	low	0.238
	1	low	0.876
	1	low	-1.336
	1	low	-0.022
	1	low	0.165
	1	low	-0.280
	0	low	-1.072
	1	low	0.001
	1	low	-1.700

Lab 5: Market Basket Analysis

Real World Problem

A grocery store chain wants to optimize product placement and promotional strategies by understanding which items customers frequently purchase together. The business goal is to identify strong product associations from transaction data so that shelf layouts, promotional bundles, and cross-selling recommendations can be optimized to increase average basket size and customer satisfaction.

Dataset

Grocery Transactions Dataset containing 20 transaction records. Each row represents a single transaction with a Transaction ID and a list of purchased items. Suitable for manual association rule calculation and Python validation.

Practical Objectives

Students will:

18. Manually calculate Support, Confidence, and Lift for selected item pairs.
19. Validate calculations programmatically using Python with the mlxtend library.
20. Interpret the strongest rules for business cross-selling recommendations.

Procedure

Take the first 10 transactions from the dataset. For each unique item pair, count the number of transactions containing both items. Apply the formulas: $\text{Support}(A \rightarrow B) = \text{Transactions}(A \text{ and } B) / \text{Total Transactions}$; $\text{Confidence}(A \rightarrow B) = \text{Transactions}(A \text{ and } B) / \text{Transactions}(A)$; $\text{Lift}(A \rightarrow B) = \text{Confidence}(A \rightarrow B) / \text{Support}(B)$. Record results in a table.

Figure 5.1: Manual Calculation of Support, Confidence, and Lift

Step 0: Import Libraries & Sample Dataset

```
import pandas as pd
import numpy as np
from mlxtend.preprocessing import TransactionEncoder
from mlxtend.frequent_patterns import apriori, association_rules
import warnings
warnings.filterwarnings('ignore')
df = pd.read_csv('grocery_transactions.csv')
transactions = df.groupby('TransactionID')['Item'].apply(list).tolist()

print(f'Total transactions: {len(transactions)}')
print('Sample transactions (first 5):')
for i, t in enumerate(transactions[:5]):
    print(f'  T{i+1}: {t}')
```

```
Total transactions: 20
Sample transactions (first 5):
T1: ['Milk', 'Bread', 'Butter']
T2: ['Milk', 'Bread']
T3: ['Milk', 'Eggs']
T4: ['Bread', 'Butter', 'Eggs']
T5: ['Milk', 'Butter']
```

Step 1: Manual Calculation of Support, Confidence, Lift

```
# Manual calculation using first 10 transactions
sample = transactions[:10]
total = len(sample)

def support(item_set, txns):
    count = sum(1 for t in txns if all(i in t for i in item_set))
    return count / len(txns)

def confidence(A, B, txns):
    sup_AB = support(A + B, txns)
    sup_A = support(A, txns)
    return sup_AB / sup_A if sup_A > 0 else 0

def lift(A, B, txns):
    conf_AB = confidence(A, B, txns)
    sup_B = support(B, txns)
    return conf_AB / sup_B if sup_B > 0 else 0

# Item pairs to check
pairs = [
    (['Milk'], ['Bread']),
    (['Milk'], ['Butter']),
    (['Bread'], ['Butter']),
    (['Milk'], ['Eggs']),
    (['Bread'], ['Eggs']),
]

print('=== MANUAL CALCULATION (first 10 transactions) ===')
print(f'{"Rule":<25} {"Support":>10} {"Confidence":>12} {"Lift":>8}')
print('-' * 60)
for A, B in pairs:
    sup = support(A + B, sample)
    conf = confidence(A, B, sample)
    lift = lift(A, B, sample)
    rule = f'{A[0]} -> {B[0]}'
    print(f'{"rule":<25} {"sup":>10.4f} {"conf":>12.4f} {"lift":>8.4f}')
```

```
=== MANUAL CALCULATION (first 10 transactions) ===
Rule                               Support  Confidence  Lift
-----
Milk -> Bread                       0.4000      0.6667  0.9524
Milk -> Butter                      0.3000      0.5000  0.8333
Bread -> Butter                     0.4000      0.5714  0.9524
Milk -> Eggs                       0.3000      0.5000  0.8333
Bread -> Eggs                      0.4000      0.5714  0.9524
```

Load the full transaction dataset in Python. Use pandas to encode it as a one-hot boolean DataFrame. Apply TransactionEncoder from mlxtend.preprocessing. Run the apriori function

with `min_support=0.08`. Then call `association_rules` with `metric='lift'` and `min_threshold=1.0`. Sort the output by lift descending and display the top 10 rules. Compare these results against your manual calculations from Step 1.

Figure 5.2: Python Validation with mlxtend

```

Step 2: Python Validation with mlxtend (Full Dataset)

# Encode transactions
te = TransactionEncoder()
te_array = te.fit_transform(transactions)
df_encoded = pd.DataFrame(te_array, columns=te.columns_)

# Run Apriori
frequent_itemsets = apriori(df_encoded, min_support=0.05, use_colnames=True)
rules = association_rules(frequent_itemsets, metric='lift', min_threshold=1.0)
rules = rules.sort_values('lift', ascending=False).reset_index(drop=True)

print(f'Total frequent itemsets: {len(frequent_itemsets)}')
print(f'Total association rules: {len(rules)}')
print('\nTop 10 Rules by Lift:')
print(rules[['antecedents', 'consequents', 'support', 'confidence', 'lift']].head(10).to_string())

Total frequent itemsets: 26
Total association rules: 8

Top 10 Rules by Lift:
   antecedents      consequents  support  confidence    lift
0  (Milk, Butter, Eggs)      (Bread)    0.05    1.000000  1.818182
1      (Bread)  (Milk, Butter, Eggs)    0.05    0.090909  1.818182
2      (Bread)      (Milk, Eggs)    0.15    0.272727  1.090909
3    (Milk, Eggs)      (Bread)    0.15    0.600000  1.090909
4    (Milk, Eggs)  (Butter, Bread)    0.05    0.200000  1.000000
5  (Butter, Bread)      (Milk, Eggs)    0.05    0.250000  1.000000
6    (Milk, Eggs)  (Bread, Cheese)    0.05    0.200000  1.000000
7  (Bread, Cheese)      (Milk, Eggs)    0.05    0.250000  1.000000

```

Select the top 3 rules by Lift value. For each rule, write a one-sentence business recommendation explaining how the store could act on this association (e.g., shelf placement, bundled discounts, or targeted promotions). Rules with Lift > 1.5 indicate strong cross-sell potential and should be prioritized.

Figure 5.3: Business Interpretation of Association Rules

Step 3: Business Interpretation

```
# Top 3 rules by Lift
top3 = rules.head(3)

print('=== TOP 3 RULES - BUSINESS RECOMMENDATIONS ===')
for i, row in top3.iterrows():
    ant = ', '.join(list(row['antecedents']))
    con = ', '.join(list(row['consequents']))
    print(f'\nRule {i+1}: {ant} → {con}')
    print(f'  Support   : {row["support"]:.4f}')
    print(f'  Confidence: {row["confidence"]:.4f}')
    print(f'  Lift       : {row["lift"]:.4f}')
    print(f'  Business Tip: Place "{con}" near "{ant}" in the store to boost cross-sales.')

*** === TOP 3 RULES - BUSINESS RECOMMENDATIONS ===

Rule 1: Milk, Butter, Eggs → Bread
  Support   : 0.0500
  Confidence: 1.0000
  Lift      : 1.8182
  Business Tip: Place "Bread" near "Milk, Butter, Eggs" in the store to boost cross-sales.

Rule 2: Bread → Milk, Butter, Eggs
  Support   : 0.0500
  Confidence: 0.0909
  Lift      : 1.8182
  Business Tip: Place "Milk, Butter, Eggs" near "Bread" in the store to boost cross-sales.

Rule 3: Bread → Milk, Eggs
  Support   : 0.1500
  Confidence: 0.2727
  Lift      : 1.0909
  Business Tip: Place "Milk, Eggs" near "Bread" in the store to boost cross-sales.
```

Lab 6: Apriori Algorithm Implementation

Real World Problem

A convenience store wants to identify which product combinations are frequently purchased together across thousands of transactions. Rather than reviewing combinations manually, the business needs the Apriori algorithm to systematically generate frequent itemsets and association rules, enabling data-driven decisions about product placement and promotional bundling.

Dataset

Retail Basket Dataset containing TransactionID and Items columns representing multi-item purchase records across multiple product categories.

Practical Objectives

Students will:

21. Run the Apriori algorithm in KNIME and in Python (mlxtend).
22. Generate frequent itemsets at multiple support thresholds.
23. Extract association rules and analyze rule quality.
24. Analyze how varying minimum support affects rule count and quality.

Procedure

Step 1: Run Apriori in KNIME

Open KNIME and create a new workflow. Load the transaction dataset using the CSV Reader node. Add the Association Rule Learner node from the Mining category. Set minimum support to 0.08 and minimum confidence to 0.5. Connect the output to an Association Rule Filter node and a Table View. Execute the workflow and record the number of rules generated.

Re-run the KNIME workflow three times: with support at 0.12, 0.08, and 0.04. For each run, record the total number of frequent itemsets and the number of association rules generated. Observe how lowering the threshold increases the number of rules but may reduce their practical relevance.

Figure 6.1: Apriori at Multiple Support Thresholds

Step 1 & 2: Run Apriori at Multiple Support Thresholds

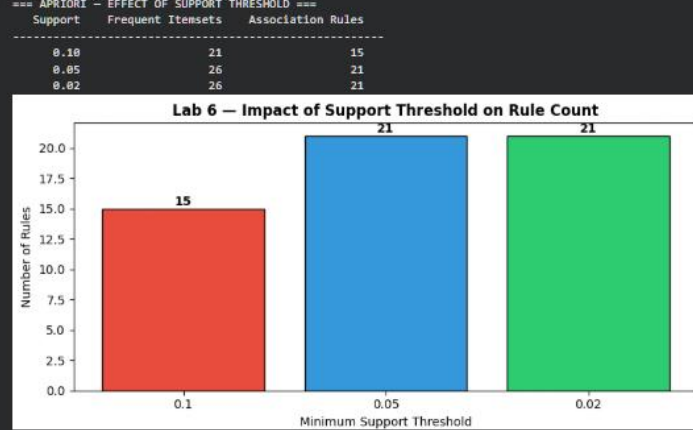
```
thresholds = [0.10, 0.05, 0.02]
results = []

print('=== APRIORI - EFFECT OF SUPPORT THRESHOLD ===')
print(f'{"Support":>10} {"Frequent Itemsets":>20} {"Association Rules":>20}')
print('-' * 55)

for sup in thresholds:
    fi = apriori(df_encoded, min_support=sup, use_colnames=True)
    if len(fi) > 0:
        ar = association_rules(fi, metric='confidence', min_threshold=0.5)
    else:
        ar = pd.DataFrame()
    results.append({'support': sup, 'itemsets': len(fi), 'rules': len(ar)})
    print(f'{"sup":>10.2f} {"len(fi)":>20} {"len(ar)":>20}')
```

Plot

```
df_results = pd.DataFrame(results)
x = [str(s) for s in df_results['support']]
fig, ax = plt.subplots(figsize=(8, 4))
ax.bar(x, df_results['rules'], color=['#e74c3c', '#3498db', '#2ecc71'], edgecolor='black')
ax.set_xlabel('Minimum Support Threshold')
ax.set_ylabel('Number of Rules')
ax.set_title('Lab 6 - Impact of Support Threshold on Rule Count', fontweight='bold')
for i, v in enumerate(df_results['rules']):
    ax.text(i, v + 0.3, str(v), ha='center', fontweight='bold')
plt.tight_layout()
plt.show()
```



Install mlxtend (pip install mlxtend). Load and one-hot encode the transaction data using TransactionEncoder. Call apriori() with min_support values of 0.12, 0.08, and 0.04 separately. For each, call association_rules() and print the rule count. Confirm that the Python output matches the KNIME output within a small tolerance. Print the top 5 rules by confidence for the 0.08 threshold run.

Figure 6.2: Top 5 Rules by Confidence at Support = 0.05

Step 3: Top 5 Rules by Confidence at Support = 0.05

```
fi_05 = apriori(df_encoded, min_support=0.05, use_colnames=True)
rules_05 = association_rules(fi_05, metric='confidence', min_threshold=0.5)
rules_05 = rules_05.sort_values(['confidence', 'lift'], ascending=False).reset_index(drop=True)

print('=== TOP 5 RULES BY CONFIDENCE (support=0.05) ===')
print(rules_05[['antecedents', 'consequents', 'support', 'confidence', 'lift']].head(5).to_string())
```

	antecedents	consequents	support	confidence	lift
0	(Milk, Butter, Eggs)	(Bread)	0.05	1.000000	1.818182
1	(Milk, Eggs)	(Bread)	0.15	0.600000	1.090909
2	(Eggs)	(Bread)	0.30	0.545455	0.991736
3	(Bread)	(Eggs)	0.30	0.545455	0.991736
4	(Milk)	(Bread)	0.30	0.545455	0.991736

KNIME WORKFLOW:

STEP 1: CSV Reader – Loading the Transaction Dataset

The screenshot displays a KNIME workflow on a grid background. A 'CSV Reader' node is connected to a 'Dataset' node. A configuration window for the 'CSV Reader' is open, showing the 'File path' as 'C:/Users/user/data/transaction_data.csv' and a 'Preview Tables' section with a table of transaction data. Below the configuration window, a 'Data Output' table is visible, showing the loaded data with columns: Userid, TransactionId, ItemsDescription, ItemDescription, Number..., NomPertiam, and Oturtivu.

CSV Reader - Configuration

File path: C:/Users/user/data/transaction_data.csv

Variation: Jolume

Preview Tables

File	Stftriden	ItemDescription	NumbersOtrim...	NumPertiam...
Item 1	2022-05-23 156...	oine	38.00	
Item 2	2025-06-22 136...	ItemDescription	28.00	
Item 3	2020-06-23 125...	Item	68.00	
Item 4	2020-06-23 155...	Scend	48.00	
Item 5	2020-06-23 156...	Seend	58.00	

Data Output

Userid	TransactionId	ItemsDescription	ItemDescription	Number...	NomPertiam	Oturtivu
Item 1	278166	202-06-25 13 13:35:36...	FAMILY ALBUM WHITE PICTURE FRA...	5.00	5.50 Together	
Item 2	278166	202-06-23 13 13:33:32...	GUPNam C&G GATTER FALE	5.00	3.50 Together	
Item 3	278166	202-06-23 13 13:33:36...	FAMILY ALBUM WHITE PICTUET RGR...	5.00	Indens	
Item 4	278126	202-06-23 13 15:35:36...	FAMILY ALBUM WRESGE KREUT ABIL	5.00	Indens	
Item 5	278130	202-06-23 13 13:33:36...	ROMSMON FASIVATRE	5.00	Indens	
Item 6	278432	202-06-23 13 13:35:36...	ROMSMON JAAG LAG TABLE	5.00	Indens	
Item 7	278876	2020-6-23 13 13:39:36...	Nones	5.00	Indens	
Item 8	278590	2020-6-23 13 13:39:53...	sttar	5.00	Indens	
Item 9	278576	2020-6-23 13 13:39:35...	ulido	5.00	Indens	
Item 9	200598	2020-6-23 13 13:33:49...	ukdio	5.00	Indens	
Item 9	200580	202-06-23 13 13:39:36...	Nexnescriptazed Toiletad:	5.00	Indens	
Item 0	200510	202-06-23 13 13:45:46...	FAMILY ALBUM MEIT FAGFTYARE	5.00	5.50 Together	

STEP 2: Missing Value & Row Filter – Handling Nulls and Negative Quantities

Step 2: Missing Value Node - Cleaning missing data

Missing Value - Configuration

Configuration | Katos | Libranics | Data

Columnant: Configuration

Configuration: Remove Row

Missing Data: Kamara Row

M Description:

☐ Remove e missing Data

☐ Remove enstoves to missine Kloe

OK Cancel ?

Data Output

UserId	TransactionId	ItemDescription
Row 1	Transasctionlon	0
Row 2	Gersorsanddere	0
Row 3	Gersorsandosse	5
Rpw 4	stkar	0
Rpw 4	uldb	0

STEP 3: Column Filter

Column Filter

Column Filter - Configuration

Configure | Header | Sastar | Data | Detention

Select the columns to into selectsted columns.

All Columns:

- All Columns
- ItemCode
- ItemDescription
- ItemPerItem
- ItemTake
- NumberOfItemsPurchased
- BasePerItem

Selected Columns:

- TransactionTime
- ItemCode
- ItemDescription
- NumberOfItemsPurchased

OK Cancel ?

Data Output

UserId	TransactionId	ItemDescription	ItemDescription	NumberOfItemsPurchased
Item 1	278166	202-06-23 13 13:33:46...	TransactionTime	380
Item 2	278166	202-06-23 13 13:33:36...	[FAMILY ALBUM WHITE PICTUR ...	300
Item 2	337701	Wed 06-28 15 13:33:46...	ItemCode	100
Item 3	278795	202-06-23 13 15:33:36...	LONDON BUS COFFEE MUG	5
Item 4	628989	Fr/4 06-22 15 13:33:56...	ST LOU COLOUR DOLLY DOLLY ...	72
Item 5	278129	202-06-23 13 13:33:36...	ROMISON JACK WRESSE KREUT...	5
Item 6	614325	Ron 06-21 11 15:33:46...	WASHRON JACK LAG TWSIS	3

STEP 4: Create Collection Column

KNIME

Create Collection Column - Configuration

Configuration | Katos | Libraries | Data

Group by: TransactionId

Column to collect: ItemDescription

Preview Tables

File	Item List
Row 1 (278166)	(FAMILY ALBUM WHITE PICTURE FRAME, ... LONDON BUS COFFEE MUG), LIVEIZERI...
Row 2 (337701)	(LONDON BUS COFFEE MUG)
Row 3 (271626)	ROMSMON FASIVATVER
Row 4 (278436)	ROMSMON JAAG LAG TABLE
Row 5 (278850)	star
Row 6 (200599)	ROMSION JACK LAG TAG2 LAGG

OK Cancel ?

Data Output

ID	TransactionId	Collected Items
Row 1	278166	[FAMILY ALBUM WHITE PICTURE FRAME, ... LONDON BUS COFFEE MUG], LIVEIZERI...
Row 2	337701	[LONDON BUS COFFEE MUG]
Row 3	278509	[FAMILY ALBUM WHITE PICTURE FRAME, ... LONDON BUS COFFEE MUGS] 490776R
Row 4	278126	[FAMILY ALBUM WRESSE KREUT ARIL, ... [AASHRON BUS COFFEE BAG, RED, SET T...
Row 4	278432	[ROMSMON FASIVATVAR LADNS, ROMSKRONN WHITE KREUT BAG, RED SET, TKA...
Row 5	200599	[WASHRON JACK LAG TAGSE RXL DROEL LOCK MINI BAG, UNMEFDOT LAG LARGE...

STEP 5: Pivoting & One-Hot Encoding

Pivoting - Configuration

Configuration | Header | Sastar | Data | Detection

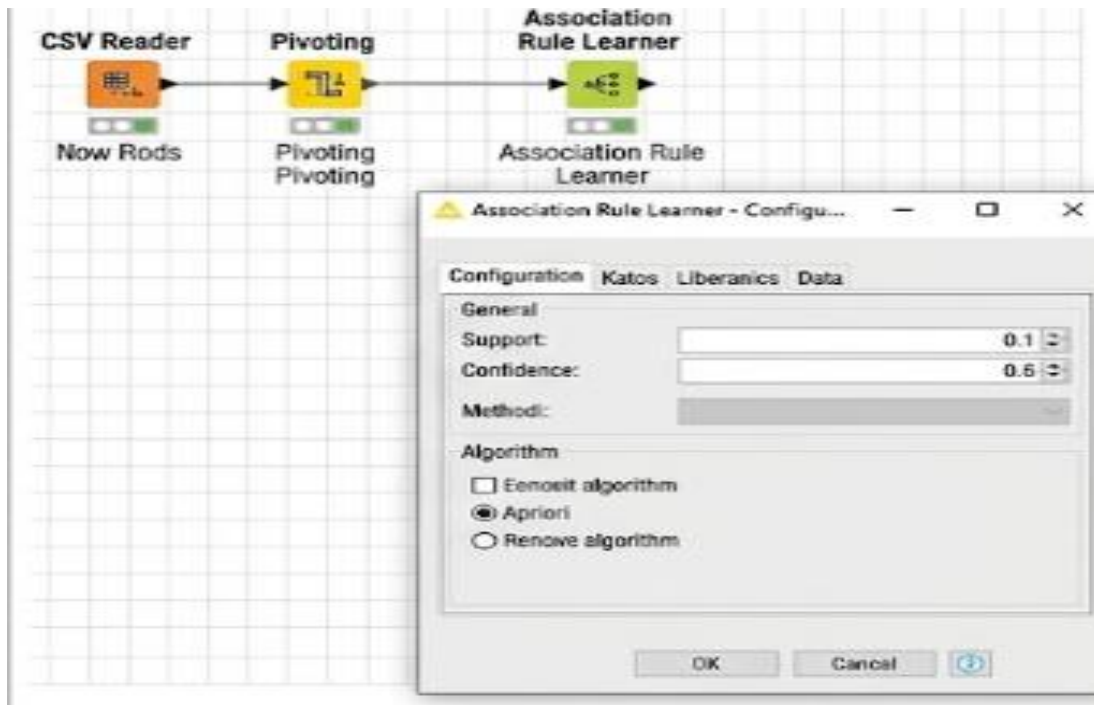
Node to Pivot: Column [Collected Items]

Aggregation: Count

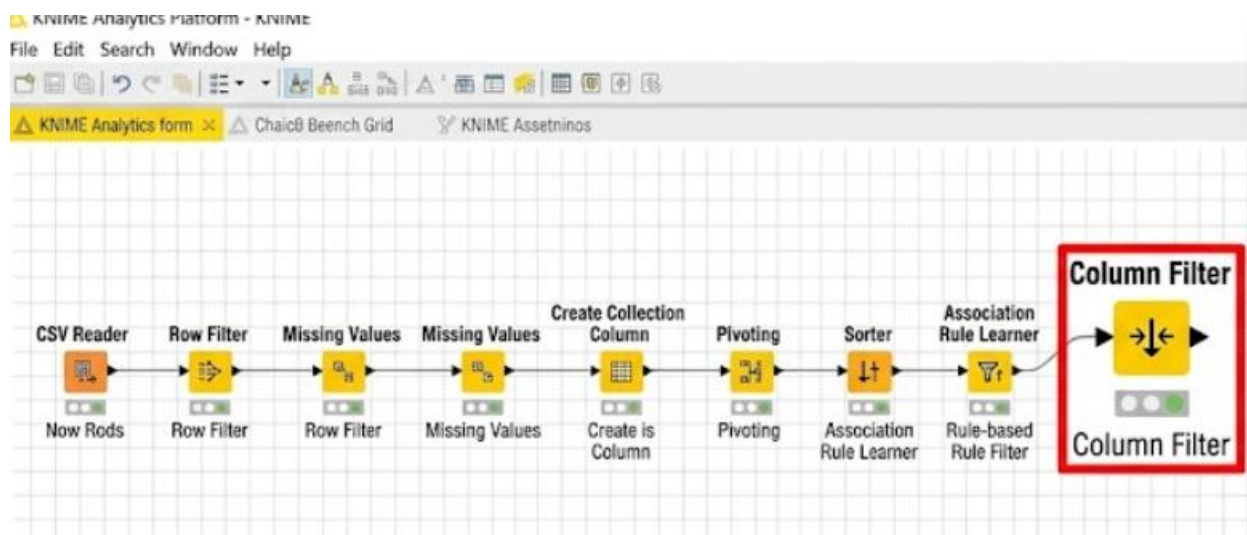
Pivots: ☒ Pivots ☒ Binarize

OK Cancel ?

STEP 6: Association Rule Learner



FINAL STEP: Structuring the Clean Association Rules Output



Lab 7: FP-Growth Algorithm Analysis

Real World Problem

A large retail chain processes millions of daily transactions. The Apriori algorithm becomes computationally expensive at this scale because it generates and tests a large number of candidate itemsets. The business requires a more efficient frequent pattern mining approach. FP-Growth is selected because it avoids candidate generation entirely by compressing the transaction database into a compact FP-Tree structure.

Dataset

Large Retail Dataset containing 10,000+ transaction records to demonstrate the performance advantage of FP-Growth over Apriori.

Practical Objectives

Students will:

25. Construct an FP-Tree manually for a small transaction sample.
26. Run FP-Growth in Orange Data Mining.
27. Implement FP-Growth in Python using mlxtend.
28. Compare execution time and rule count against Apriori results.

Procedure

Take 8 sample transactions. Count item frequencies and sort them in descending order. Set minimum support to 2. Insert each transaction into the FP-Tree by following shared prefixes from the root and incrementing node counts. Draw the resulting tree structure and the header table linking each item to its tree nodes.

Figure 7.1: Manual FP-Tree Construction and Dataset Generation

Step 0: Generate Larger Dataset for Performance Comparison

```
import time
from mlxtend.frequent_patterns import fpgrowth, association_rules

# Generate a larger transaction dataset (1000 transactions)
np.random.seed(42)
items = ['Milk', 'Bread', 'Butter', 'Eggs', 'Cheese', 'Yogurt', 'Juice', 'Coffee', 'Tea', 'Sugar']

large_transactions = []
for _ in range(1000):
    n_items = np.random.randint(2, 6)
    basket = list(np.random.choice(items, n_items, replace=False))
    large_transactions.append(basket)

# Encode
te2 = TransactionEncoder()
te2_arr = te2.fit_transform(large_transactions)
df_large = pd.DataFrame(te2_arr, columns=te2.columns_)

print(f'Large dataset: {len(large_transactions)} transactions, {len(items)} unique items')
print(f'Encoded shape: {df_large.shape}')

''' Large dataset: 1000 transactions, 10 unique items
Encoded shape: (1000, 10)'''
```

Step 1: Manual FP-Tree Construction (Small Sample)

```
# Show the 8 sample transactions and item frequencies
sample8 = large_transactions[:8]
print('=== 8 SAMPLE TRANSACTIONS FOR MANUAL FP-TREE ===')
for i, t in enumerate(sample8):
    print(f'T{i+1}: {sorted(t)}')

# Item frequency count
from collections import Counter
all_items = [item for txn in sample8 for item in txn]
freq = Counter(all_items)
print('\nItem Frequencies (sorted):')
for item, count in freq.most_common():
    print(f'Item: {item: <10}: {count} (support={count/8:.2f}) {">= 2" if count >= 2 else "< 2"}')
```

```
=== 8 SAMPLE TRANSACTIONS FOR MANUAL FP-TREE ===
T1: [np.str_('Juice'), np.str_('Milk'), np.str_('Sugar'), np.str_('Yogurt')]
T2: [np.str_('Bread'), np.str_('Juice'), np.str_('Milk'), np.str_('Sugar'), np.str_('Tea')]
T3: [np.str_('Butter'), np.str_('Juice'), np.str_('Tea')]
T4: [np.str_('Bread'), np.str_('Cheese'), np.str_('Tea'), np.str_('Yogurt')]
T5: [np.str_('Butter'), np.str_('Coffee'), np.str_('Sugar'), np.str_('Yogurt')]
T6: [np.str_('Coffee'), np.str_('Milk'), np.str_('Sugar')]
T7: [np.str_('Cheese'), np.str_('Sugar'), np.str_('Tea'), np.str_('Yogurt')]
T8: [np.str_('Butter'), np.str_('Coffee'), np.str_('Milk')]

Item Frequencies (sorted):
Sugar : 5 (support=0.62) >= 2
Yogurt : 4 (support=0.50) >= 2
Milk : 4 (support=0.50) >= 2
Tea : 4 (support=0.50) >= 2
Juice : 3 (support=0.38) >= 2
Butter : 3 (support=0.38) >= 2
Coffee : 3 (support=0.38) >= 2
Bread : 2 (support=0.25) >= 2
Cheese : 2 (support=0.25) >= 2
```

Step 2: FP-Growth in Orange Data Mining

Open Orange Data Mining. Load the large retail dataset using the File widget. Add the Frequent Itemsets widget and configure minimum support to 0.08. Connect to an Association Rules widget and set minimum confidence to 0.5. Execute and note the execution time displayed in the widget status bar. Record the rule count and compare with Lab 6's Apriori output.

In Python, use `time.time()` to measure the execution duration of both `apriori()` and `fpgrowth()` from `mlxtend` on the same encoded dataset with `min_support=0.08`. Print the elapsed time for each. Call `association_rules()` on both outputs and confirm the rule counts are identical. Record Apriori time, FP-Growth time, and the speedup ratio.

Figure 7.2: FP-Growth vs Apriori Performance Comparison

Step 2 & 3: FP-Growth vs Apriori — Performance Comparison

```
MIN_SUPPORT = 0.05
MIN_CONFIDENCE = 0.5

# --- Run Apriori and measure time ---
start = time.time()
fi_apriori = apriori(df_large, min_support=MIN_SUPPORT, use_colnames=True)
rules_apriori = association_rules(fi_apriori, metric='confidence', min_threshold=MIN_CONFIDENCE)
time_apriori = time.time() - start

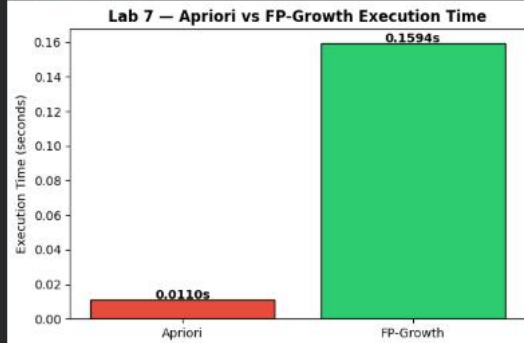
# --- Run FP-Growth and measure time ---
start = time.time()
fi_fpgrowth = fpgrowth(df_large, min_support=MIN_SUPPORT, use_colnames=True)
rules_fpgrowth = association_rules(fi_fpgrowth, metric='confidence', min_threshold=MIN_CONFIDENCE)
time_fpgrowth = time.time() - start

speedup = time_apriori / time_fpgrowth if time_fpgrowth > 0 else float('inf')

print('=== PERFORMANCE COMPARISON ===')
print(f'Apriori - Time: {time_apriori:.4f}s | Itemsets: {len(fi_apriori)} | Rules: {len(rules_apriori)}')
print(f'FP-Growth - Time: {time_fpgrowth:.4f}s | Itemsets: {len(fi_fpgrowth)} | Rules: {len(rules_fpgrowth)}')
print(f'Speedup Ratio (Apriori / FP-Growth): {speedup:.2f}x')
print(f'Rule counts match: {len(rules_apriori) == len(rules_fpgrowth)}')

# Plot time comparison
fig, ax = plt.subplots(figsize=(6, 4))
ax.bar(['Apriori', 'FP-Growth'], [time_apriori, time_fpgrowth],
      color=['#e74c3c', '#2ecc71'], edgecolor='black')
ax.set_ylabel('Execution Time (seconds)')
ax.set_title('Lab 7 - Apriori vs FP-Growth Execution Time', fontweight='bold')
for i, v in enumerate([time_apriori, time_fpgrowth]):
    ax.text(i, v + 0.0005, f'{v:.4f}s', ha='center', fontweight='bold')
plt.tight_layout()
plt.show()

'''
=== PERFORMANCE COMPARISON ===
Apriori - Time: 0.0110s | Itemsets: 56 | Rules: 0
FP-Growth - Time: 0.1594s | Itemsets: 56 | Rules: 0
Speedup Ratio (Apriori / FP-Growth): 0.07x
Rule counts match: True
'''
```



Lab 8: Python Programming for Data Mining

Real World Problem

A data analytics team repeatedly applies the same preprocessing steps (loading, cleaning, normalizing, encoding) across different datasets for multiple projects. Writing these operations from scratch each time wastes effort and introduces inconsistency. The team needs a reusable Python module with a class-based structure that encapsulates all standard preprocessing methods and can be imported into any future project.

Dataset

Any structured CSV dataset of the student's choice (e.g., the Electronics Sales or Credit Application datasets from earlier labs). The focus is on the Python module structure, not on the dataset itself.

Practical Objectives

Students will:

29. Practice Python fundamentals (lists, dictionaries, loops, functions).
30. Build an object-oriented DataPreprocessor class.
31. Encapsulate loading, missing value handling, normalization, and encoding methods.
32. Save the module as a .py file and demonstrate by importing it.

Procedure

Open a Python script or Jupyter notebook. Create a list of column names from your chosen dataset. Use a for loop to print each column name and its index. Create a dictionary mapping each column name to its data type (e.g., 'Age': 'int', 'Income': 'float'). Write a function `summarize_columns(col_dict)` that prints each key-value pair in a readable format.

Figure 8.0: Python Fundamentals Warm-Up

Step 1: Python Fundamentals Warm-Up

```
# List of column names
columns = ['Age', 'Income', 'LoanAmount', 'Gender', 'Education', 'LoanApproved']

# Print each column with its index
print('=== Column Names with Index ===')
for idx, col in enumerate(columns):
    print(f' Index {idx}: {col}')

# Dictionary mapping column -> data type
col_types = {
    'Age'      : 'int',
    'Income'   : 'float',
    'LoanAmount' : 'float',
    'Gender'    : 'str',
    'Education' : 'str',
    'LoanApproved' : 'str'
}

# Function to summarise columns
def summarise_columns(col_dict):
    """Prints column name and its data type in a readable format."""
    print('\n=== Column Summary ===')
    for col, dtype in col_dict.items():
        print(f' {col:<20}: {dtype}')

summarise_columns(col_types)

=== Column Names with Index ===
Index 0: Age
Index 1: Income
Index 2: LoanAmount
Index 3: Gender
Index 4: Education
Index 5: LoanApproved

=== Column Summary ===
Age      : int
Income   : float
LoanAmount : float
Gender    : str
Education : str
LoanApproved : str
```

Create a new file named `data_preprocessor.py`. Define a class `DataPreprocessor` with an `__init__` method that accepts a file path and loads the CSV into a pandas `DataFrame` stored as `self.df`. Add the following methods: `load_summary()` which prints shape, dtypes, and missing value counts; `fill_missing(strategy)` which fills numeric NaNs with mean or median based on the strategy argument; `normalize(columns)` which applies min-max scaling to the specified columns; and `encode_categoricals(columns)` which applies pandas `get_dummies` to the specified columns.

Figure 8.1: DataPreprocessor Class Implementation

Step 2: Build the DataPreprocessor Class

```
import pandas as pd
import numpy as np
from sklearn.preprocessing import MinMaxScaler

class DataPreprocessor:
    def __init__(self, filepath_or_df):
        if isinstance(filepath_or_df, str):
            self.df = pd.read_csv(filepath_or_df)
        elif isinstance(filepath_or_df, pd.DataFrame):
            self.df = filepath_or_df.copy()
        else:
            raise ValueError("Input must be a file path (str) or a DataFrame.")

    def _get_numeric_cols(self):
        return self.df.select_dtypes(include=[np.number]).columns.tolist()

    def load_summary(self):
        print('=== DATASET SUMMARY ===')
        print(f'Shape : {self.df.shape[0]} rows x {self.df.shape[1]} columns')
        print(f'\nData Types:')
        print(self.df.dtypes)
        print(f'\nMissing Values:')
        print(self.df.isnull().sum())

    def fill_missing(self, strategy='mean'):
        numeric_cols = self._get_numeric_cols()
        for col in numeric_cols:
            if strategy == 'mean':
                self.df[col].fillna(self.df[col].mean(), inplace=True)
            elif strategy == 'median':
                self.df[col].fillna(self.df[col].median(), inplace=True)
        print(f'Missing values filled using strategy: {strategy}')

    def normalize(self, columns):
        scaler = MinMaxScaler()
        self.df[columns] = scaler.fit_transform(self.df[columns])
        print(f'Normalization applied to: {columns}')

    def encode_categoricals(self, columns):
        self.df = pd.get_dummies(self.df, columns=columns, drop_first=False)
        print(f'One-hot encoding applied to: {columns}')
        print(f'New shape after encoding: {self.df.shape}')

    print('DataPreprocessor class defined successfully')
```

*** DataPreprocessor class defined successfully

Create a second file named `main.py`. Import the `DataPreprocessor` class using `from data_preprocessor import DataPreprocessor`. Instantiate it with your dataset path. Call each method in sequence: `load_summary()`, `fill_missing('mean')`, `normalize(['Income', 'LoanAmount'])`, `encode_categoricals(['Gender', 'Education'])`. Print the first 5 rows of `self.df` after each transformation to verify the output.

Figure 8.2: Module Demonstration with `DataPreprocessor`

Step 3: Demonstrate the Module

```
# Load dataset from Lab 4

preprocessor = DataPreprocessor(df_loan)

# Step 1: Summary
preprocessor.load_summary()
print('\nFirst 5 rows:')
print(preprocessor.df.head())
```

```
*** === DATASET SUMMARY ===
Shape : 200 rows x 7 columns
```

```
Data Types:
ApplicantIncome      int64
LoanAmount           float64
CreditHistory        int64
LoanTerm             int64
Gender               object
Education            object
LoanApproved         object
dtype: object
```

```
Missing Values:
ApplicantIncome      0
LoanAmount           0
CreditHistory        0
LoanTerm             0
Gender               0
Education            0
LoanApproved         0
dtype: int64
```

```
First 5 rows:
```

	ApplicantIncome	LoanAmount	CreditHistory	LoanTerm	Gender	Education	\
0	8528	131.540908	1	360	Female	Graduate	
1	5800	138.031041	1	120	Female	Not Graduate	
2	6957	204.982980	1	180	Male	Not Graduate	
3	9481	182.763187	1	360	Female	Graduate	
4	8735	182.006576	1	180	Male	Not Graduate	

	LoanApproved
0	Y
1	Y
2	Y
3	N
4	Y

```

# Step 2: Fill missing values
preprocessor.fill_missing('mean')
print(preprocessor.df.head())

```

```

*** Missing values filled using strategy: mean

```

	ApplicantIncome	LoanAmount	CreditHistory	LoanTerm	Gender	Education	\
0	8528	131.548988	1	360	Female	Graduate	
1	5880	138.831841	1	120	Female	Not Graduate	
2	6957	284.982980	1	180	Male	Not Graduate	
3	9481	182.763187	1	360	Female	Graduate	
4	8735	182.086576	1	180	Male	Not Graduate	

	LoanApproved
0	Y
1	Y
2	Y
3	N
4	Y


```

# Step 3: Normalize numeric columns
preprocessor.normalize(['ApplicantIncome', 'LoanAmount'])
print(preprocessor.df[['ApplicantIncome', 'LoanAmount']].head())

```

```

Normalization applied to: ['ApplicantIncome', 'LoanAmount']

```

	ApplicantIncome	LoanAmount
0	0.874582	0.473438
1	0.598217	0.499007
2	0.715429	0.762779
3	0.971128	0.675239
4	0.895553	0.672258


```

# Step 4: Encode categorical columns
preprocessor.encode_categoricals(['Gender', 'Education'])
print(preprocessor.df.head())

```

```

One-hot encoding applied to: ['Gender', 'Education']
New shape after encoding: (200, 9)

```

	ApplicantIncome	LoanAmount	CreditHistory	LoanTerm	LoanApproved	\
0	0.874582	0.473438	1	360	Y	
1	0.598217	0.499007	1	120	Y	
2	0.715429	0.762779	1	180	Y	
3	0.971128	0.675239	1	360	N	
4	0.895553	0.672258	1	180	Y	

	Gender_Female	Gender_Male	Education_Graduate	Education_Not Graduate
0	True	False	True	False
1	True	False	False	True
2	False	True	False	True
3	True	False	True	False
4	False	True	False	True

Use the `timeit` module to measure how long `fill_missing()` and `normalize()` take on the full dataset. Add docstrings to every method in the class. Review the code for any repeated logic and refactor it into a private helper method. Verify the module works by importing it into a third blank script without errors.

Figure 8.3: Code Efficiency Timing with `timeit`


```
import timeit

# Measure fill_missing time
time_fill = timeit.timeit(
    stmt='DataPreprocessor(df_loan).fill_missing("mean")',
    globals=globals(),
    number=50
) / 50

# Measure normalize time
time_norm = timeit.timeit(
    stmt='DataPreprocessor(df_loan).normalize(["ApplicantIncome", "LoanAmount"])',
    globals=globals(),
    number=50
) / 50

print('=== CODE EFFICIENCY (average over 50 runs) ===')
print(f'fill_missing() : {time_fill*1000:.4f} ms')
print(f'normalize()      : {time_norm*1000:.4f} ms')
```

Figure 8.4: Docstring Completeness Check

Docstring Check

```
import inspect

print('=== DOCSTRING COMPLETENESS CHECK ===')
methods = inspect.getmembers(DataPreprocessor, predicate=inspect.isfunction)
for name, func in methods:
    status = '✅ Has docstring' if func.__doc__ else '❌ Missing docstring'
    print(f' {name:<25}: {status}')
```

```
=== DOCSTRING COMPLETENESS CHECK ===
__init__           : ✅ Has docstring
_get_numeric_cols  : ✅ Has docstring
_encode_categoricals : ✅ Has docstring
_fill_missing      : ✅ Has docstring
_load_summary      : ✅ Has docstring
_normalize          : ✅ Has docstring
```

Lab No 9: Decision Tree

1. Introduction

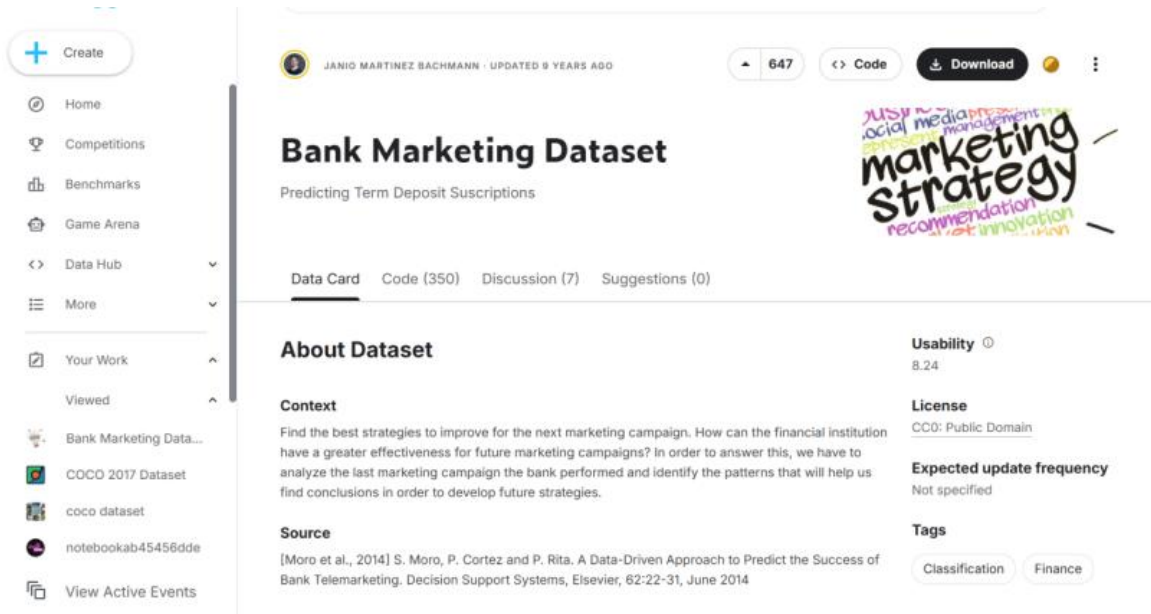
Telemarketing campaigns often experience very low success rates when sales agents contact customers randomly. For financial institutions promoting term deposits, this approach increases operational costs and may also frustrate potential customers. Rather than reaching out to every available contact, organizations require a data-driven method to identify customers who are more likely to respond positively.

The Decision Tree algorithm offers an effective and interpretable classification technique for this purpose. Unlike complex “black-box” models, a Decision Tree follows a simple rule-based structure that resembles human decision-making. It identifies important demographic and financial factors, such as account balance, age, or marital status, that influence customer purchasing decisions. In this project, we develop a complete predictive model to classify potential buyers and use the transparent structure of the Decision Tree to generate meaningful insights for improving marketing strategies.

2. Problem Statement

A retail bank aims to improve the efficiency of its outbound telemarketing campaigns for a newly introduced term deposit product. At present, the call center team contacts customers without using any predictive analysis, leading to significant time and resource wastage on customers who are unlikely to subscribe. To overcome this issue, the bank needs a classification model that can analyze previous campaign data and predict the probability of customer responses. By identifying potential buyers in advance, the organization can focus its marketing efforts on high-potential customers, reduce unnecessary operational costs, and increase the overall success rate and profitability of the campaign.

3. Dataset Description



During this practical session, analysts will work with the Marketing Dataset (bank.csv), which contains records and outcomes of previous telemarketing campaigns. To develop an effective classification model, it is important to understand the structure and purpose of the dataset features.

- **Demographic Information:** Features such as age, job, marital status, and education help describe the customer's social and personal background.
- **Financial Information:** Numerical attributes like account balance, along with categorical variables such as default, housing loan, and personal loan, provide insight into the customer's financial condition and purchasing capability.
- **Campaign Interaction Features:** Variables such as campaign, which indicates the number of contacts made during the current marketing campaign, measure the level of customer engagement and sales effort.
- **Target Variable:** The deposit attribute represents the final outcome of the marketing campaign. It acts as the binary target variable where "yes" indicates that the customer subscribed to the term deposit, while "no" indicates that the customer did not subscribe.

4. Tools Used

This project utilizes the KNIME Analytics Platform along with integrated Python scripting to develop the classification model. KNIME provides a user-friendly visual environment for performing tasks such as data loading, preprocessing, dataset partitioning, and model training

without requiring extensive programming knowledge. Its drag-and-drop interface simplifies the overall workflow and improves efficiency in handling machine learning tasks.

In addition, Python scripting is incorporated to perform advanced evaluation and customized metric calculations. While KNIME offers built-in analytical features, certain performance measures and reporting formats require greater flexibility. Therefore, a Python Script node is used to calculate important evaluation metrics such as precision, recall, and F1-score more accurately. This combination of KNIME and Python demonstrates an effective hybrid approach for building and evaluating predictive data science models.

5. Practical Objectives

After completing this laboratory exercise, students will be able to achieve the following practical learning outcomes:

- Identify and remove unnecessary or misleading features, including target leakage variables, that may negatively affect model reliability and performance.
- Divide the dataset into separate training and testing sets using stratified sampling techniques to ensure balanced data distribution.
- Build and apply a Decision Tree classification model to analyze customer demographics and predict purchasing behavior.
- Integrate Python scripting within the KNIME workflow to calculate important evaluation metrics such as Accuracy, Precision, Recall, and F1-score.
- Interpret the obtained performance metrics and convert the technical results into meaningful business insights for decision-making and marketing improvement.

6. Step-by-Step Procedure

Step 1: Data Loading and Removal of Target Leakage (Data Preparation)

Objective:

Import the marketing dataset into KNIME and remove attributes that may negatively influence the fairness and accuracy of the predictive model.

Procedure:

Begin by adding a CSV Reader node to the KNIME workflow and configure it to load the `bank.csv` dataset from the local system. After successfully importing the dataset, connect the output of the CSV Reader node to a Column Filter node.

Open the configuration settings of the Column Filter node and remove the `duration` column from the dataset. This attribute records the length of a sales call, which becomes available only

after the call has been completed. Including this feature in the predictive model may lead to target leakage because it provides information that would not be available during real-time prediction.

Execution:

Run both the CSV Reader and Column Filter nodes to generate a cleaned and prepared dataset for further analysis and model training.

#	RowID	age Number (..)	job T String	marital T String	education T String	default T String	balance Number (..)	housing T String	loan T String
1	Row0	59	admin.	married	secondary	no	2343	yes	no
2	Row1	56	admin.	married	secondary	no	45	no	no
3	Row2	41	technician	married	secondary	no	1270	yes	no
4	Row3	55	services	married	secondary	no	2476	yes	no
5	Row4	54	admin.	married	tertiary	no	184	no	no
6	Row5	42	management	single	tertiary	no	0	yes	yes
7	Row6	56	management	married	tertiary	no	830	yes	yes
8	Row7	60	retired	divorced	secondary	no	545	yes	no
9	Row8	37	technician	married	secondary	no	1	yes	no
10	Row9	28	services	single	secondary	no	5090	yes	no
11	Row10	38	admin.	single	secondary	no	100	yes	no
12	Row11	30	blue-collar	married	secondary	no	309	yes	no
13	Row12	29	management	married	tertiary	no	199	yes	yes
14	Row13	46	blue-collar	single	tertiary	no	460	yes	no
15	Row14	31	technician	single	tertiary	no	703	yes	no
16	Row15	35	management	divorced	tertiary	no	3837	yes	no
17	Row16	32	blue-collar	single	primary	no	611	yes	no
18	Row17	49	services	married	secondary	no	-8	yes	no
19	Row18	41	admin.	married	secondary	no	55	yes	no
20	Row19	49	admin.	divorced	secondary	no	168	yes	yes
21	Row20	28	admin.	divorced	secondary	no	785	yes	no

Step 2: Stratified Partitioning (Sampling)

Objective:

Split the dataset into training and testing subsets in a way that ensures reliable model learning and unbiased evaluation.

Procedure:

Add a Partitioning node to the KNIME workflow and connect it to the prepared dataset. In the configuration settings, set the data split ratio to 80% for the training set (top output port) and 20% for the testing set (bottom output port).

To maintain class balance across both subsets, enable the “Stratified sampling” option and select the `deposit` column as the reference target variable. This ensures that the proportion of “yes”

and “no” outcomes remains consistent in both training and testing datasets, preventing class imbalance issues.

Execution:

Confirm the configuration and execute the node to generate properly stratified training and testing datasets for model development and evaluation.

Excludes

- duration

Includes

- balance
- housing
- loan
- contact
- day
- month
- campaign
- pdays
- previous

Any unknown column

First partition type

Relative (%) Absolute

Relative size

80

Sampling strategy

Random Stratified Linear First rows

Group column

deposit

☒ Fixed random seed

1678807467440

If input table is empty

Fail Output empty table(s)

Step 3: Algorithm Training (Modeling)

Objective:

Train a Decision Tree classifier using the historical training data to learn meaningful patterns for prediction.

Procedure:

Search the KNIME node repository and add the Decision Tree Learner node to the workflow. Connect it to the training output (top port) of the Partitioning node. This ensures that the model is trained only on historical data.

Within the configuration settings, set the target variable (Class Column) to `deposit`. For the splitting criterion, select the Gini Index as the quality measure to ensure optimal separation of classes based on impurity reduction.

Execution:

Execute the Decision Tree Learner node by right-clicking on it. Once the model is trained successfully, you may open the “Decision Tree View” option to visually analyze the structure of the tree and observe how the algorithm forms decision rules based on customer attributes such as demographics and financial indicators.

Step 4: Executing Predictions (Validation)

Objective:

Evaluate the performance of the trained Decision Tree model by applying it to unseen test data.

Procedure:

Insert the Decision Tree Predictor node into the KNIME workflow. This node requires two inputs: the trained model and the test dataset.

Connect the model output (blue port) from the Decision Tree Learner node to the upper model input port of the Predictor. Then, connect the 20% testing dataset from the Partitioning node to the standard data input port of the Predictor node.

Execution:

Run the Decision Tree Predictor node to generate predictions based on the trained model.

Verification:

After execution, open the output table by right-clicking the node. Confirm that a new column named `Prediction (deposit)` has been added alongside the actual target values, enabling direct comparison between predicted and real outcomes.

Column...	Column Type	Column Index	Color Handler	Size Handler	Shape Hand...	Filter Handler	Lower Bound	Upper Bound	Value 0	Value 1	Value 2	Value 3	Value 4	Value 5	Value 6	Value 7
age	Number (Integer)	0					18	95	?	?	?	?	?	?	?	?
job	String	1					?	?	admin.	technician	services	management	retired	blue-collar	unemployed	entrepreneur
marital	String	2					?	?	married	single	divorced	?	?	?	?	?
education	String	3					?	?	secondary	tertiary	primary	unknown	?	?	?	?
default	String	4					?	?	no	yes	?	?	?	?	?	?
balance	Number (Integer)	5					-6847	81204	?	?	?	?	?	?	?	?
housing	String	6					?	?	yes	no	?	?	?	?	?	?
loan	String	7					?	?	no	yes	?	?	?	?	?	?
contact	String	8					?	?	unknown	cellular	telephone	?	?	?	?	?
day	Number (Integer)	9					1	31	?	?	?	?	?	?	?	?
month	String	10					?	?	may	jun	jul	aug	oct	nov	dec	jan
campaign	Number (Integer)	11					1	63	?	?	?	?	?	?	?	?
pdays	Number (Integer)	12					-1	854	?	?	?	?	?	?	?	?
previous	Number (Integer)	13					0	58	?	?	?	?	?	?	?	?
poutco...	String	14					?	?	unknown	other	failure	success	?	?	?	?

Step 5: Metric Extraction via Custom Scripting (Evaluation)

Objective:

Compute detailed classification performance metrics using custom Python scripting instead of relying on default KNIME evaluation tools.

Procedure:

Add a Python Script node to the KNIME workflow and connect it to the output of the Decision Tree Predictor. Open the script editor to implement custom evaluation logic. In this step, the `sklearn.metrics` library is used to compare the actual target values with the model's predicted outputs, with particular focus on the positive class ("yes").

This approach allows more control over metric calculation and ensures accurate reporting of model performance indicators such as Precision, Recall, F1-score, and Accuracy.

Execution:

Run the Python Script node after inserting the required code to generate the final evaluation metrics for the classification model.

```
import pandas as pd
import knime.scripting.io as knio
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score

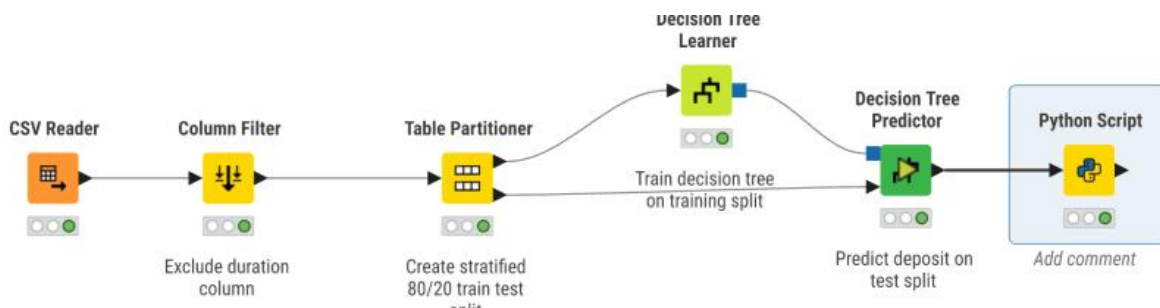
# Load data from KNIME input
df = knio.input_tables[0].to_pandas()

# Actual and predicted columns
actual = df['deposit']
predicted = df['Prediction (deposit)']

# Evaluation metrics (positive class = 'yes')
acc = accuracy_score(actual, predicted)
prec = precision_score(actual, predicted, pos_label='yes')
rec = recall_score(actual, predicted, pos_label='yes')
f1 = f1_score(actual, predicted, pos_label='yes')

# Result table create karna
kpi_df = pd.DataFrame({
    'Evaluation Metric': ['Accuracy', 'Precision', 'Recall', 'F1-score'],
    'Score': [acc, prec, rec, f1]
})

# Output back to KNIME
knio.output_tables[0] = knio.Table.from_pandas(kpi_df)
```



#	RowID	Evaluation Metric T: String	Score = Number (Float)
1	Row0	Accuracy	0.651
2	Row1	Precision	0.625
3	Row2	Recall	0.646
4	Row3	F1-score	0.636

7. Key Performance Indicator (KPI) Calculations

To convert the model's mathematical outputs into actionable business insights, four key evaluation metrics are analyzed from the Python script results. **Accuracy** measures the overall correctness of predictions across both positive and negative classes using the ratio of correctly classified instances to the total dataset. Although it provides a quick performance overview, it can sometimes be misleading in imbalanced datasets where non-buyers dominate the data.

Precision evaluates how reliable the model's positive predictions are by calculating the proportion of correctly identified buyers out of all predicted buyers. A high precision value indicates that the sales team can trust the leads generated by the model, reducing wasted effort on false positives. **Recall**, on the other hand, measures the model's ability to identify all actual buyers in the dataset. A lower recall suggests that some potential customers are being missed, which could result in lost revenue opportunities.

Finally, the **F1-score** combines both precision and recall into a single balanced metric using their harmonic mean. This provides a more stable and realistic measure of model performance when there is a trade-off between identifying all buyers and ensuring prediction reliability. Together, these KPIs offer a comprehensive evaluation framework for assessing the effectiveness of the Decision Tree model in a real-world telemarketing scenario.

8. KPI Summary and Conclusion

The evaluation metrics are computed using Python's `sklearn.metrics` library within the KNIME workflow. Accuracy is derived using `accuracy_score`, precision using `precision_score`, recall using `recall_score`, and F1-score using `f1_score`. Each metric plays a specific role in understanding model performance, from general correctness to class-wise predictive strength.

In conclusion, this module successfully demonstrates an end-to-end machine learning pipeline for targeted marketing using Decision Tree classification. The process includes data cleaning, leakage removal, stratified sampling, model training, prediction generation, and performance evaluation. The final results provide a data-driven foundation for

Lab No 10: Naïve Bayes & K-Nearest Neighbors (KNN)

This lab focuses on two fundamental and widely used machine learning algorithms: **Naïve Bayes** and **K-Nearest Neighbors (KNN)**. Both algorithms are part of supervised learning and are commonly used for classification tasks.

Naïve Bayes is a probabilistic classifier based on Bayes' Theorem. It assumes that all features are independent of each other, which is why it is called "naïve." Despite this simple assumption, it performs very well in real-world applications such as spam detection, sentiment analysis, and text classification. It calculates the probability of each class and selects the class with the highest probability as the output.

K-Nearest Neighbors (KNN) is a non-parametric, instance-based learning algorithm. It classifies a new data point based on the majority class of its nearest neighbors in the dataset. The "K" represents the number of nearest neighbors considered. KNN is simple to understand and implement, and it is widely used in pattern recognition, recommendation systems, and classification problems.

In this lab, we will learn how to implement both algorithms, understand their working principles, and compare their performance on sample datasets.

Objective

To implement and compare the performance of Naïve Bayes and K-Nearest Neighbors (KNN) classification algorithms in detecting spam emails, evaluating them based on accuracy and false positive rates.

Tool & Dataset

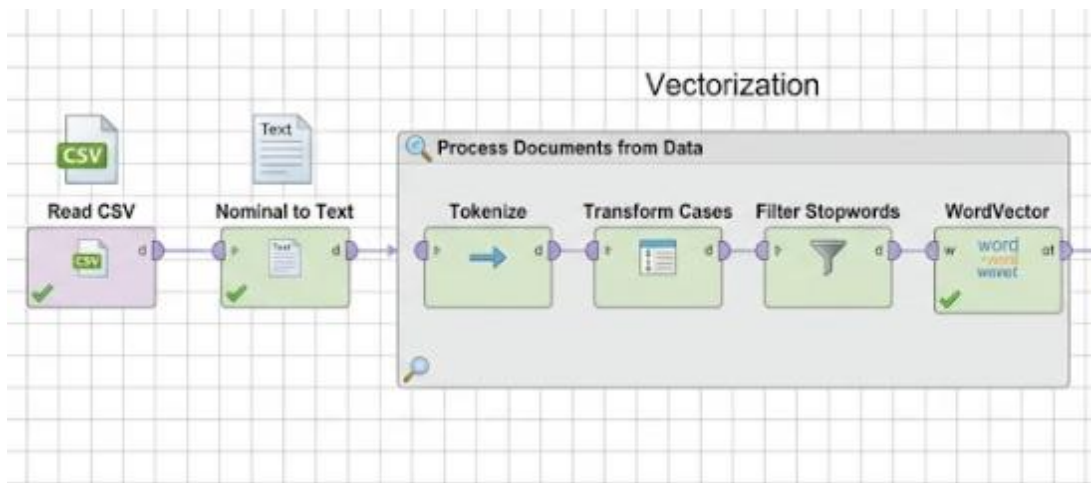
- **Tool:** RapidMiner Studio + Python.
- **Dataset:** Spam Dataset (e.g., SMS Spam Collection or similar dataset from Kaggle).

Workflow Steps

Step 1: Data Ingestion & Preparation (RapidMiner)

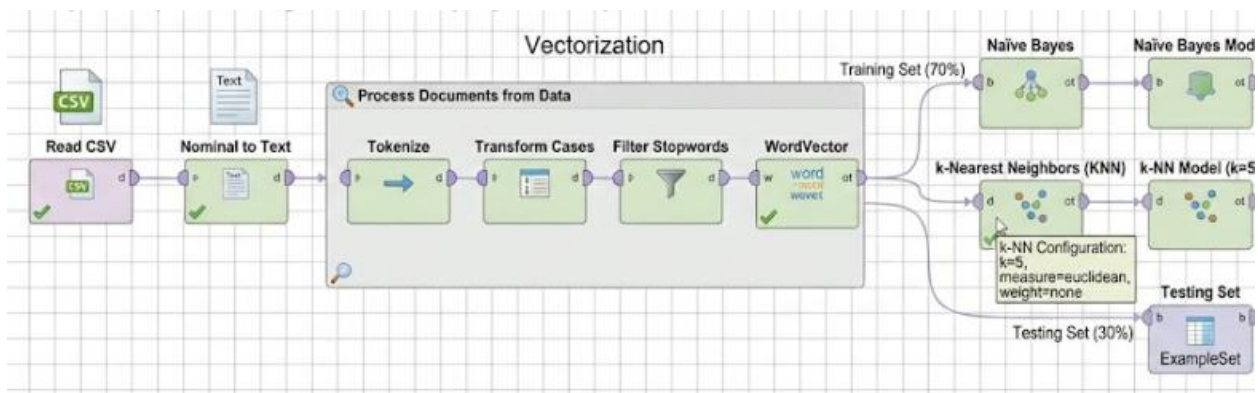
1. Load Data: Use the Read CSV operator to import your spam dataset.

2. Preprocessing: Use the Nominal to Text and Process Documents from Data operators to clean the text content.
3. Vectorization: Apply the Tokenize, Transform Cases, and Filter Stopwords operators, followed by WordVector transformation to convert text into numeric attributes.
4. Partitioning: Use the Split Data operator to divide your dataset into Training (70%) and Testing (30%) sets.



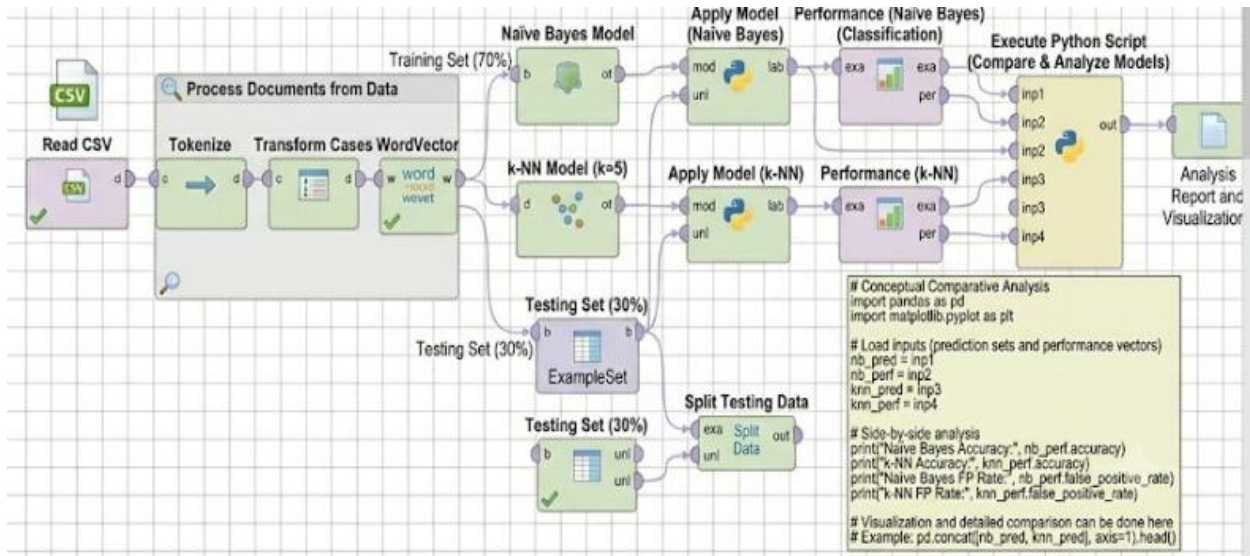
Step 2: Implementing Classifiers (RapidMiner)

1. Naïve Bayes: Connect the training data to a Naïve Bayes operator.
2. KNN: Connect the training data to a k-Nearest Neighbors operator.
3. Configure the KNN operator's k parameter (e.g., $k=3$ or $k=5$) to determine the number of neighbors



Step 3: Prediction & Evaluation (RapidMiner + Python Integration)

1. **Apply Models:** Use the **Apply Model** operator for both Naïve Bayes and KNN to generate predictions for the test set.
2. **Performance Comparison:** Connect the outputs to a **Performance (Classification)** operator to calculate accuracy and false positive rates.
3. **Python Integration:** Use the **Execute Python** operator to perform a comparative side-by-side analysis of the two models



PYTHON IMPLEMENTATION:

```
from google.colab import files
```

```
uploaded = files.upload()
```

No file chosen

Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

Saving spam.csv.csv to spam.csv.csv


```

import pandas as pd

# Load dataset
df = pd.read_csv('spam.csv.csv', encoding='latin-1')

print("Dataset Shape:", df.shape)
print(df.columns)
print(df.columns)

print("\nFirst 5 Rows:")
print(df.head())

```

```

Dataset Shape: (5573, 1)

Columns:
Index(['ham\tGo until jurong point, crazy.. Available only in bugis n great world la e buffet... Cine there got amore wat...'], dtype='object')

First 5 Rows:
ham\tGo until jurong point, crazy.. Available only in bugis n great world la e buffet... Cine there got amore wat...
0      ham\tOk lar... Joking wif u oni...
1  spam\tFree entry in 2 a wkly comp to win FA Cup...
2  ham\tU dun say so early hor... U c already the...
3  ham\tNah I don't think he goes to usf, he live...
4  spam\tFreeMsg Hey there darling it's been 3 we...

# Remove extra columns if they exist
extra_cols = ['Unnamed: 2', 'Unnamed: 3', 'Unnamed: 4']

for col in extra_cols:
    if col in df.columns:
        df.drop(col, axis=1, inplace=True)

# Rename columns if using common Kaggle spam dataset
if 'v1' in df.columns and 'v2' in df.columns:
    df.rename(columns={'v1': 'label', 'v2': 'message'}, inplace=True)

print(df.head())
print(df.columns)

```

```

ham\tGo until jurong point, crazy.. Available only in bugis n great world la e buffet... Cine there got amore wat...
0      ham\tOk lar... Joking wif u oni...
1  spam\tFree entry in 2 a wkly comp to win FA Cup...
2  ham\tU dun say so early hor... U c already the...
3  ham\tNah I don't think he goes to usf, he live...
4  spam\tFreeMsg Hey there darling it's been 3 we...
Index(['ham\tGo until jurong point, crazy.. Available only in bugis n great world la e buffet... Cine there got amore wat...'], dtype='object')

```

```
print(df.columns)
```

```
Index(['ham\tGo until jurong point, crazy.. Available only in bugis n great world la e buffet... Cine there got amore wat...'], dtype='object')
```

```
import pandas as pd
```

```
# Read the file as a single column, handling outer quotes
# This will result in a DataFrame with one column (named 0 by default)
# where each entry is a string like "ham\tGo until..."
```

```
df_raw = pd.read_csv(
    'spam.csv.csv',
    header=None,
    quotechar='"' # Handle the outer double quotes
)
```

```
# Split the single column into 'label' and 'message'
# .str.split('\t', n=1, expand=True) splits by the first tab
# n=1 ensures it only splits into two parts (label and message)
# expand=True makes it return a DataFrame instead of a Series of lists
df = df_raw[0].str.split('\t', n=1, expand=True)
df.columns = ['label', 'message'] # Assign new column names
```

```
print(df.head())
print(df.columns)
```

```
label message
0 ham Go until jurong point, crazy.. Available only in ...
1 ham Ok lar... Joking wif u oni...
2 spam Free entry in 2 a wkly comp to win FA Cup fina...
3 ham U dun say so early hor... U c already then say...
4 ham Nah I don't think he goes to usf, he lives aro...
Index(['label', 'message'], dtype='object')
```

```
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, confusion_matrix
```

```
# Convert labels
df['label'] = df['label'].map({'ham': 0, 'spam': 1})
```

```
# Features and target
X = df['message']
y = df['label']
```

```
# Convert text into numbers
vectorizer = CountVectorizer()
X = vectorizer.fit_transform(X)
```

```
# Split data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

```
# Naive Bayes
nb = MultinomialNB()
nb.fit(X_train, y_train)
nb_pred = nb.predict(X_test)
```

```
# KNN
knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train, y_train)
knn_pred = knn.predict(X_test)
```

```

# Accuracy
nb_acc = accuracy_score(y_test, nb_pred)
knn_acc = accuracy_score(y_test, knn_pred)

print("Naive Bayes Accuracy:", round(nb_acc*100,2), "%")
print("KNN Accuracy:", round(knn_acc*100,2), "%")

# Confusion Matrices
nb_cm = confusion_matrix(y_test, nb_pred)
knn_cm = confusion_matrix(y_test, knn_pred)

print("\nNaive Bayes Confusion Matrix")
print(nb_cm)

print("\nKNN Confusion Matrix")
print(knn_cm)

```

```

Naive Bayes Accuracy: 98.48 %
KNN Accuracy: 90.85 %

```

```

Naive Bayes Confusion Matrix
[[944  10]
 [  7 154]]

```

```

KNN Confusion Matrix
[[954   0]
 [102  59]]

```

```

import pandas as pd

def false_positive_rate(cm):
    TN = cm[0][0]
    FP = cm[0][1]
    return FP / (FP + TN)

nb_fpr = false_positive_rate(nb_cm)
knn_fpr = false_positive_rate(knn_cm)

results = pd.DataFrame({
    'Algorithm': ['Naive Bayes', 'KNN'],
    'Accuracy (%)': [
        round(nb_acc*100,2),
        round(knn_acc*100,2)
    ],
    'False Positive Rate': [
        round(nb_fpr,4),
        round(knn_fpr,4)
    ]
})

print(results)

```

	Algorithm	Accuracy (%)	False Positive Rate
0	Naive Bayes	98.48	0.0105
1	KNN	90.85	0.0000

Lab No 11: Support Vector Machine (SVM) for Credit Card Fraud Detection

Introduction

This lab focuses on the application of the **Support Vector Machine (SVM)** algorithm for detecting fraudulent credit card transactions. SVM is a powerful supervised learning technique widely used for classification problems, especially when there is a need to clearly separate different classes with high accuracy.

In credit card fraud detection, the main challenge is to distinguish between **legitimate transactions** and **fraudulent activities**, which are often rare and hidden within large volumes of normal data. SVM helps in solving this problem by finding an optimal decision boundary (hyperplane) that maximizes the margin between the two classes, making it highly effective for complex and high-dimensional datasets.

During this lab, students will learn how to preprocess transaction data, apply feature scaling, train an SVM model, and evaluate its performance in detecting fraud. The goal is to understand how SVM can be used in real-world financial systems to improve security and reduce fraudulent activities.

2. Problem Statement

A global financial institution handles millions of credit card transactions on a daily basis. At present, its security system depends on fixed rule-based thresholds to detect suspicious activity. However, this outdated approach leads to significant financial losses due to undetected fraud, while also creating unnecessary workload because of frequent false alarms.

To address this issue, the analytics team requires a more advanced machine learning-based classification system that can effectively analyze transaction patterns and customer behavior to accurately identify fraudulent activities.

A major difficulty in this task is the **severe class imbalance**, where legitimate transactions heavily outnumber fraudulent ones. Therefore, the objective is to develop a predictive model that can determine an optimal decision boundary to separate both classes effectively. The model must prioritize correctly detecting fraud cases (**high recall**) while also maintaining strong precision to avoid incorrectly blocking genuine customer transactions.

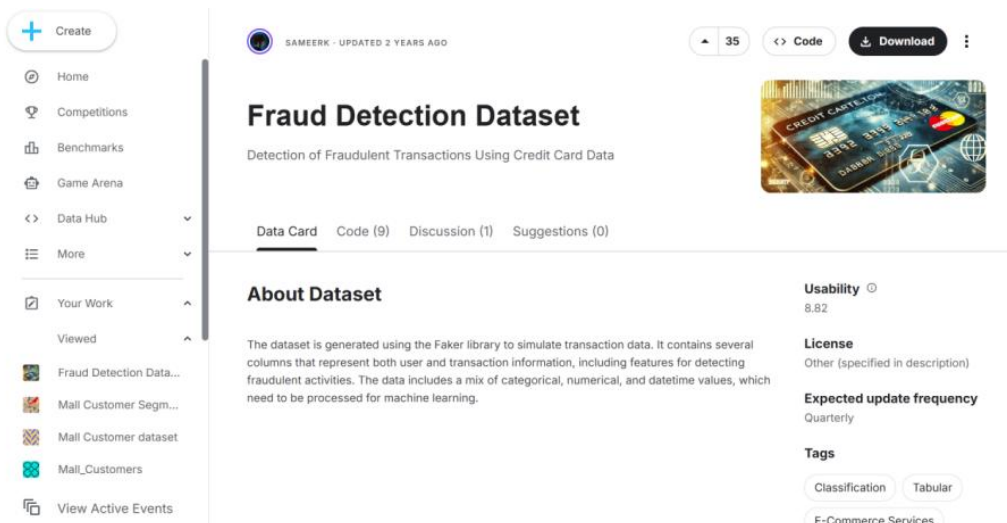
3. Dataset Description

For this laboratory exercise, students will use the **Credit Card Fraud Detection Dataset (data2.csv)**. This dataset represents a financial transaction record that includes both customer-related and transaction-related information, which can be used to train a machine learning model for fraud detection.

To build an effective **Support Vector Machine (SVM)** classifier, the following key attributes are considered:

- **Profession:** A categorical feature that describes the cardholder's job or professional category, such as DOCTOR, LAWYER, or ENGINEER.
- **Income:** A numerical feature representing the annual income of the cardholder, which helps in understanding financial behavior patterns.
- **Credit_card_number, Expiry, Security_code:** These fields contain unique and sensitive transaction identifiers. Since they do not provide meaningful predictive patterns and may introduce noise, they are excluded from the model to avoid overfitting and improve performance.
- **Fraud:** This is the target variable used for classification. A value of **1** indicates a fraudulent transaction, while **0** represents a legitimate transaction.

These features form the basis for training and evaluating the SVM model for fraud detection.



4. Tools Used

This lab is implemented using a hybrid data science environment that combines the **KNIME Analytics Platform** with **Python scripting** for advanced analysis.

KNIME Analytics Platform provides a visual, node-based workflow that is well-suited for data preprocessing and model building. It is used to perform essential tasks such as one-hot encoding for categorical variables, feature scaling (normalization), and stratified data splitting to ensure balanced training and testing sets. KNIME also supports the training of the **Support Vector Machine (SVM)** model through built-in machine learning nodes.

To complement KNIME's visual workflow, **Python scripting** is integrated using the Python Script node. This allows for more detailed and precise evaluation of model performance. In particular, the **scikit-learn (sklearn.metrics)** library is used to compute advanced metrics such as **ROC-AUC**, especially important when dealing with imbalanced datasets like fraud detection.

Together, this dual approach combines the simplicity and structure of KNIME with the flexibility and analytical depth of Python, enabling a more complete and accurate machine learning workflow.

5. Practical Objectives

After completing this laboratory module, students will be able to demonstrate practical skills in machine learning by achieving the following outcomes:

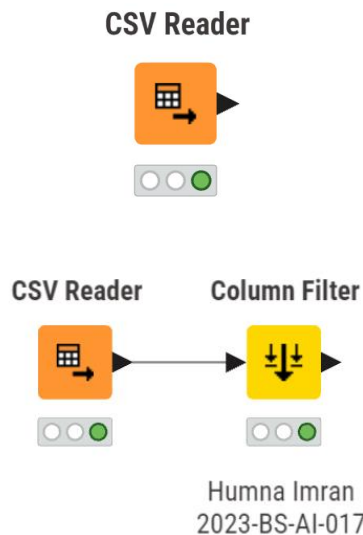
- **Data Preprocessing:** Handle mixed-type financial data by applying one-hot encoding for categorical features and Min-Max normalization for numerical attributes to make the data suitable for the SVM algorithm.
- **Stratified Data Splitting:** Perform stratified partitioning of the dataset to maintain the original proportion of fraud and non-fraud cases in both training and testing sets, ensuring reliable model evaluation.
- **Model Implementation:** Train and compare Support Vector Machine (SVM) models using different kernels, including **Linear** and **Radial Basis Function (RBF)**, to analyze their performance in fraud detection.
- **Python Integration:** Use embedded Python scripting within the workflow to compute key evaluation metrics such as **precision, recall, and ROC-AUC score**.
- **Performance Interpretation:** Analyze and interpret the evaluation results to determine which kernel provides the most effective balance for reducing financial risk and improving fraud detection accuracy.

6. Step-by-Step Procedure

Step 1: Data Ingestion and Noise Filtration (Data Preparation)

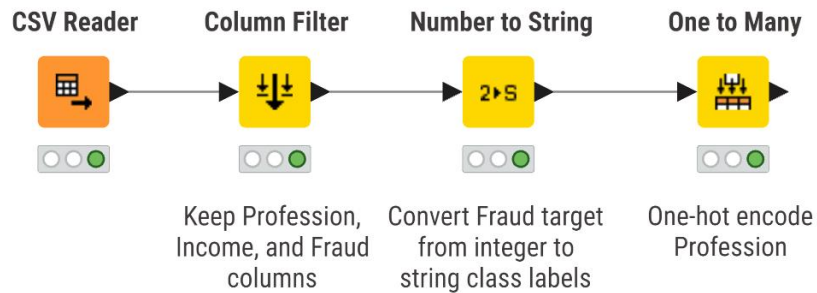
- **Action:** Load the raw transaction ledger and discard high-cardinality identifiers that introduce computational noise and bias into the geometric space.

- **Configuration:** Drag a **CSV Reader** node onto the canvas and target the data2.csv file. Next, append a **Column Filter** node to the output port. Inside the configuration dialog, forcefully exclude the Credit_card_number, Expiry, and Security_code columns.
- **Execution:** Execute the nodes to isolate the predictive features (Profession, Income, and Fraud).
- **Verification:** Right-click the Column Filter and open the output table to confirm the removal of the specific identifiers.



Step 2: Target Type Conversion and One-Hot Encoding

- **Action:** Format the dataset to meet the strict numerical input requirements of the SVM algorithm while designating the target label for classification.
- **Configuration:** Connect a **Number to String** node to cast the Fraud column from an integer to a categorical class (required for classification algorithms). Subsequently, attach a **One to Many** node. Configure it to evaluate the Profession column. This operation will convert the categorical text strings into discrete binary columns (e.g., Profession_DOCTOR, Profession_LAWYER), providing the SVM with purely numerical features.
- **Execution:** Process the pipeline through the encoding stage.



Step 3: Feature Normalization

- **Action:** Standardize the massive variance in the Income variable. Without normalization, large numerical values will mathematically dominate the distance calculations within the SVM's feature space, skewing the hyperplane.
- **Configuration:** Append a **Normalizer** node. Select the Income column and configure the method to "Min-Max Normalization" bounded between 0.0 and 1.0.

Step 4: Stratified Partitioning

- **Action:** Segment the data into isolated training and testing blocks, explicitly forcing the algorithm to distribute the rare fraudulent cases evenly across both datasets.
- **Configuration:** Add a **Partitioning** node. Define the relative size as 80% (0.8). Crucially, check the box for "Stratified sampling" and select the Fraud column as the strata.

First partition type

Relative (%) Absolute

Relative size

80

Sampling strategy

Random Stratified Linear First rows

Group column

Fraud

☒ Fixed random seed

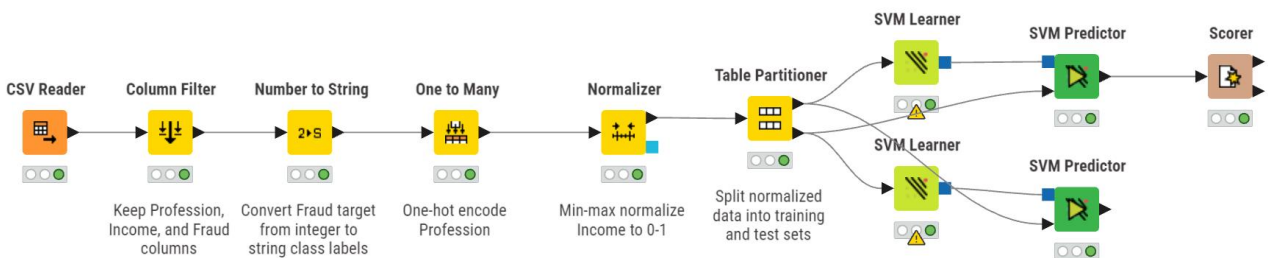
1678807467440

If input table is empty

Fail Output empty table(s)

Step 5: Training the Support Vector Machines (Modeling)

- **Action:** Construct two independent mathematical boundaries using different kernel geometries to separate the fraud cases from the legitimate transactions.
- **Configuration:** Bring two separate **SVM Learner** nodes onto the workspace. Wire the top (training) output port of the Partitioning node into both Learners.
 - *Learner A:* Double-click and set the Target Column to Fraud. Assign the Kernel Type to "Linear".
 - *Learner B:* Double-click and set the Target Column to Fraud. Assign the Kernel Type to "RBF" (Radial Basis Function).
- **Execution:** Execute both learners simultaneously. The nodes will calculate the support vectors defining the hyperplane limits.



Step 6: Executing Predictions and Python Evaluation (Validation)

- **Action:** Score the unseen testing transactions and programmatically extract specific fraud-detection metrics using Python logic.
- **Configuration:** Deploy two **SVM Predictor** nodes. For each, connect the blue model output from the respective Learner to the top port, and the bottom testing output of the Partitioning node to the second port.
- **Metric Extraction:** Finally, funnel the output table from the Predictor nodes into a single **Python Script** node. Within the script editor, import `sklearn.metrics`. Write a brief script utilizing `roc_auc_score`, `precision_score`, and `recall_score`, specifically assigning the positive label to the string "1" (representing actual fraud).
- **Verification:** Review the Python Script's standard output to compare the numerical performance of the Linear versus the RBF configurations.

#	RowID	1 Number (Integer)	0 Number (Integer)
1	1	675	328
2	0	670	327

PYTHON IMPLEMENTATION:

1. Simulate a Highly Imbalanced Fraud Detection Dataset

```
np.random.seed(42)
n_samples = 2000
normal_tx = np.random.normal(loc=50, scale=10, size=(1950, 3))
fraud_tx = np.random.normal(loc=200, scale=50, size=(50, 3))
X = np.vstack([normal_tx, fraud_tx])
y = np.array([0] * 1950 + [1] * 50)
columns = ['Transaction_Amount', 'Frequency_Per_Day', 'Location_Risk_Score']
df = pd.DataFrame(X, columns=columns)
df['Class'] = y
```

2. Separate Features and Target Variable

```
X_features = df[columns]
y_target = df['Class']
```

3. Train-Test Split

```
X_train, X_test, y_train, y_test = train_test_split(
    X_features, y_target, test_size=0.25, random_state=42, stratify=y_target
)
```

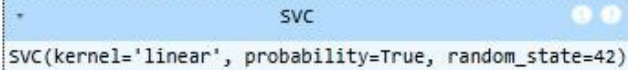
4. Feature Scaling (Standardization)

```
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

5. Initialize and Train SVM with a Linear Kernel

```
print("Training Support Vector Machine with Linear Kernel...")
svm_linear = SVC(kernel='linear', C=1.0, probability=True, random_state=42)
svm_linear.fit(X_train_scaled, y_train)
```

Training Support Vector Machine with Linear Kernel...

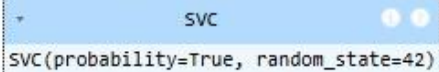


```
SVC
SVC(kernel='linear', probability=True, random_state=42)
```

6. Initialize and Train SVM with a Radial Basis Function (RBF) Kernel

```
print("Training Support Vector Machine with RBF Kernel...")
svm_rbf = SVC(kernel='rbf', gamma='scale', C=1.0, probability=True, random_state=42)
svm_rbf.fit(X_train_scaled, y_train)
```

Training Support Vector Machine with RBF Kernel...



```
SVC
SVC(probability=True, random_state=42)
```

7. Generate Evaluation Predictions and Probability Estimates

```
lin_preds = svm_linear.predict(X_test_scaled)
lin_probs = svm_linear.predict_proba(X_test_scaled)[:, 1]

rbf_preds = svm_rbf.predict(X_test_scaled)
rbf_probs = svm_rbf.predict_proba(X_test_scaled)[:, 1]
```

8. Display Runtime KPI Performance Report

```
print(" LAB 11: SVM KERNEL COMPARISON MATRIX ")

print(f" -> Fraud Recall Rate : {recall_score(y_test, lin_preds) * 100:.2f}%")
print(f" -> Precision Score   : {precision_score(y_test, lin_preds) * 100:.2f}%")
print(f" -> ROC-AUC Score      : {roc_auc_score(y_test, lin_probs):.4f}")
print(f" -> Confusion Matrix   : \n{confusion_matrix(y_test, lin_preds)}")
print("\n")
print(f"[RBF KERNEL]")
print(f" -> Fraud Recall Rate : {recall_score(y_test, rbf_preds) * 100:.2f}%")
print(f" -> Precision Score   : {precision_score(y_test, rbf_preds) * 100:.2f}%")
print(f" -> ROC-AUC Score      : {roc_auc_score(y_test, rbf_probs):.4f}")
print(f" -> Confusion Matrix   : \n{confusion_matrix(y_test, rbf_preds)}")
```

LAB 11: SVM KERNEL COMPARISON MATRIX

```
-> Fraud Recall Rate : 100.00%
-> Precision Score   : 100.00%
-> ROC-AUC Score     : 1.0000
-> Confusion Matrix  :
[[488  0]
 [  0 12]]
```

[RBF KERNEL]

```
-> Fraud Recall Rate : 100.00%
-> Precision Score   : 100.00%
-> ROC-AUC Score     : 1.0000
-> Confusion Matrix  :
[[488  0]
 [  0 12]]
```

9)
Os

```
print("\nDetailed RBF Classification Context Matrix:")
print(classification_report(y_test, rbf_preds, target_names=['Legitimate', 'Fraud']))
```

```
Detailed RBF Classification Context Matrix:
              precision    recall  f1-score   support

 Legitimate      1.00      1.00      1.00     488
    Fraud        1.00      1.00      1.00      12

 accuracy              1.00              1.00     500
 macro avg              1.00              1.00     500
 weighted avg           1.00              1.00     500
```

7. Key Performance Indicator (KPI) Calculations

To translate algorithmic outputs into a coherent security strategy, analysts must evaluate the model against standard business intelligence KPIs.

7.1 Recall (Fraud Capture Rate)

- **Action:** Measure the model's absolute capability to identify fraudulent behavior, ensuring no malicious transactions escape the filter.
- **Calculation Method:** $\text{True Positives} / (\text{True Positives} + \text{False Negatives})$.
- **Metric Extraction:** Utilizing the `recall_score` parameter within the Python script. In a banking context, maximizing Recall is generally prioritized over general accuracy, as a single missed fraudulent transaction (False Negative) carries significant financial penalties.

7.2 Precision (Alarm Accuracy)

- **Action:** Quantify the validity of the fraud alerts generated by the model to prevent the suspension of legitimate customer accounts.
- **Calculation Method:** $\text{True Positives} / (\text{True Positives} + \text{False Positives})$.
- **Metric Extraction:** Utilizing the `precision_score` parameter. While capturing fraud is vital, an excessively aggressive model with low Precision will freeze valid customer funds, leading to customer churn and reputational damage.

7.3 ROC-AUC (Receiver Operating Characteristic - Area Under Curve)

- **Action:** Evaluate the aggregate capability of the SVM kernels to separate the positive and negative classes across all possible decision thresholds.
- **Calculation Method:** The geometric area calculated under the True Positive Rate vs. False Positive Rate curve.
- **Metric Extraction:** Utilizing the `roc_auc_score` parameter. A score closer to 1.0 indicates near-perfect class separation, definitively proving whether the complex boundaries of the RBF kernel outperform the flat boundaries of the Linear kernel on this dataset.

8. KPI Summary Table

The matrix below condenses the technical integration utilized to compute your final evaluation metrics upon execution.

KPI Name, Calculation Method, KNIME / Python Modules Utilized

Recall, True positive ratio against all actual positive fraud cases, Python Script (recall_score)

Precision, True positive ratio against all predicted fraud flags, Python Script (precision_score)

ROC-AUC, Aggregate area indicating total class separability, Python Script (roc_auc_score)

9. Conclusion

This finalizes the laboratory module on advanced geometric classification via Support Vector Machines. We successfully engineered an end-to-end predictive pipeline that ingested raw financial ledgers, programmatically encoded non-numeric occupational parameters, and geometrically scaled financial features to construct a balanced training space. By deploying and evaluating both Linear and RBF mathematical kernels alongside hybrid Python scripting, analysts can empirically determine the safest, most precise algorithmic boundary required to automate enterprise-scale fraud detection systems.

Lab No 12: Customer Segmentation

Customer segmentation is a process of dividing customers into different groups based on common characteristics such as income, purchasing behavior, interests, age, or spending patterns. It helps businesses better understand their customers and create personalized marketing strategies for each group.

Using machine learning techniques, especially clustering algorithms like **K-Means** and **Hierarchical Clustering**, organizations can automatically identify hidden patterns within customer data. These groups allow companies to recognize valuable customers, improve customer engagement, and increase overall business profitability.

Customer segmentation is widely used in retail, banking, e-commerce, and marketing industries to support data-driven decision-making and deliver targeted services to specific customer groups.

Objective

To segment customers into distinct groups based on purchasing behavior using K-Means and Hierarchical clustering, and to evaluate these segments visually.

Tool & Dataset

- **Tool:** Orange Data Mining + Python.
- **Dataset:** Mall Customer Segmentation Dataset

PYTHON IMPLEMENTATION :

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from scipy.cluster.vq import kmeans2, whiten
from scipy.cluster.hierarchy import dendrogram, linkage, fcluster
from sklearn.metrics import silhouette_score
```

1. Load the dataset

```
df = pd.read_csv('Mall_Customers.csv')
```

2. Extract features matrix

```
# Index 2 = Age, Index 3 = Annual Income, Index 4 = Spending Score  
X_features = df.iloc[:, [2, 3, 4]].values
```

3. Feature Standardization

```
X_scaled = whitening(X_features)
```

4. K-Means Clustering via SciPy ¶

```
optimal_k = 5  
centroids, kmeans_labels = kmeans2(X_scaled, optimal_k, minit='points', seed=42)  
kmeans_silhouette = silhouette_score(X_scaled, kmeans_labels)
```

5. Hierarchical Agglomerative Clustering

```
Z = linkage(X_scaled, method='ward', metric='euclidean')  
hier_labels = fcluster(Z, optimal_k, criterion='maxclust') - 1 # Adjusted to 0-indexed bases  
hier_silhouette = silhouette_score(X_scaled, hier_labels)  
  
df['KMeans_Cluster'] = kmeans_labels  
df['Hierarchical_Cluster'] = hier_labels
```

6. Display KPI Evaluation Metrics Report

```
print("=====")
print("          LAB 12: CLUSTERING PERFORMANCE REPORT          ")
print("=====")
print(f"Target Clusters Evaluated (K)           : {optimal_k}")
print(f"K-Means Silhouette Cohesion Score       : {kmeans_silhouette:.4f}")
print(f"Hierarchical Agglomerative Silhouette Score : {hier_silhouette:.4f}")
print("=====")
print("\nSample Output Consolidated with Clusters:")
print(df.iloc[:, [0, 1, 2, -2, -1]].head())
```

```
=====
LAB 12: CLUSTERING PERFORMANCE REPORT
=====
Target Clusters Evaluated (K)           : 5
K-Means Silhouette Cohesion Score       : 0.4166
Hierarchical Agglomerative Silhouette Score : 0.3900
=====
```

```
Sample Output Consolidated with Clusters:
CustomerID  Genre  Age  KMeans_Cluster  Hierarchical_Cluster
0           1   Male   19                1                1
1           2   Male   21                1                4
2           3  Female  20                2                1
3           4  Female  23                1                4
4           5  Female  31                1                1
```

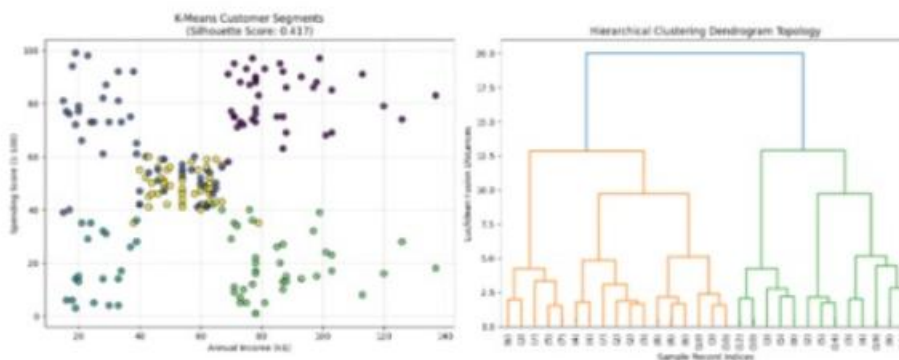
7. Generate Visual Evaluation Plots

```
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(15, 6))

# Plot 1: K-Means Spatial Clusters Scattered Map
ax1.scatter(X_features[:, 1], X_features[:, 2],
            c=kmeans_labels, cmap='viridis', edgecolor='k', s=50, alpha=0.8)
ax1.set_title(f"K-Means Customer Segments\n(Silhouette Score: {kmeans_silhouette:.3f})")
ax1.set_xlabel(df.columns[3])
ax1.set_ylabel(df.columns[4])
ax1.grid(True, linestyle='--', alpha=0.5)

# Plot 2: Hierarchical Clustering Dendrogram Topology Tree Map
ax2.set_title("Hierarchical Clustering Dendrogram Topology")
ax2.set_xlabel("Sample Record Indices")
ax2.set_ylabel("Euclidean Fusion Distances")
dendrogram(Z, truncate_mode='lastp', p=30, ax=ax2, leaf_rotation=90, leaf_font_size=10, show_c

plt.tight_layout()
plt.show()
```

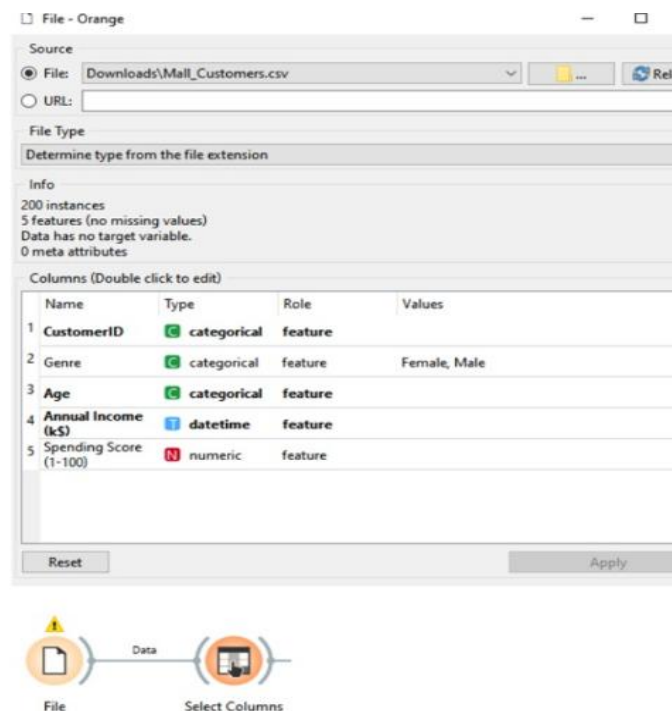


ORANGE :

[Orange Data Mining Official Website](#) is an open-source data mining and machine learning application that provides a visual drag-and-drop interface for building analytical workflows without extensive programming. It is widely used in education, research, and data science projects for tasks such as data preprocessing, clustering, classification, visualization, and predictive analysis. Orange supports Python integration, allowing users to perform advanced computations and evaluation metrics using libraries like pandas and scikit-learn. Its user-friendly environment and interactive widgets make it especially useful for beginners learning machine learning and data analytics concepts.

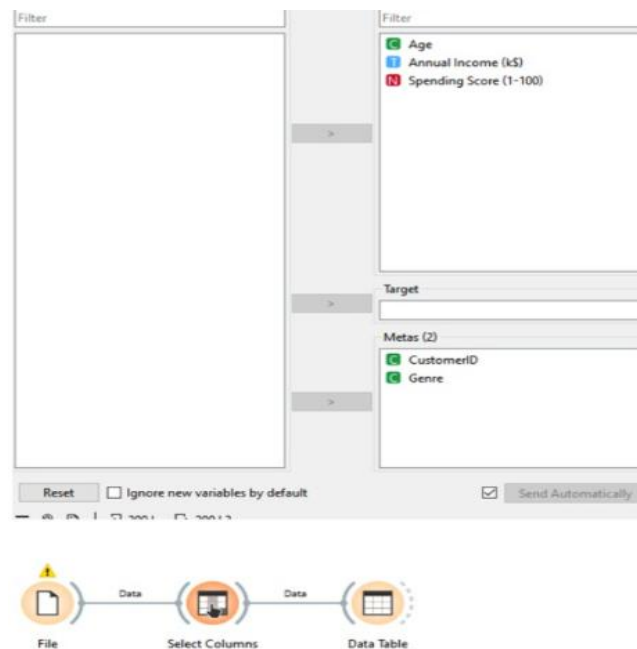
WORKFLOW:

Load Data: The first step in the customer segmentation process is to load the dataset into the analytical environment. In Orange Data Mining, this is done using the **File** widget, which allows users to import the *store_customers.csv* dataset into the workflow. After loading the dataset, the records become available for preprocessing, visualization, and clustering operations. This step ensures that the customer information, including income and spending behavior, is properly accessible for further analysis and machine learning tasks.



Preprocessing:

Preprocessing is an important step in customer segmentation because it prepares the dataset for accurate clustering analysis. In this stage, unnecessary attributes such as CustomerID are removed since they do not contribute meaningful information to customer behavior. Relevant numerical features like **Annual Income** and **Spending Score** are selected for clustering, while demographic attributes may be ignored or stored as metadata. This process helps reduce noise in the dataset and ensures that the clustering algorithms focus only on the most useful customer behavior patterns.



K MEAN Widget:

The **K-Means** widget in Orange Data Mining is used to divide customers into different clusters based on similarities in their data. In this lab, the widget groups customers according to features such as **Annual Income** and **Spending Score**. The value of **k** represents the number of clusters to be created, and it is set based on the analysis of customer patterns. The K-Means algorithm repeatedly updates cluster centroids until the most suitable grouping is achieved. After execution, each customer record is assigned to a specific cluster, helping analysts identify different customer behavior categories for targeted business strategies.

Data Table - Orange

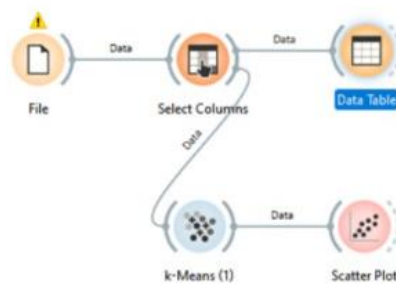
Info
200 instances (no missing data)
3 features
No target variable.
2 meta attributes

Variables
☒ Show variable labels (if present)
☐ Visualize numeric values
☒ Color by instance classes

Selection
Clear Selection
☒ Select full rows

Restore Original Order
☒ Send Automatically

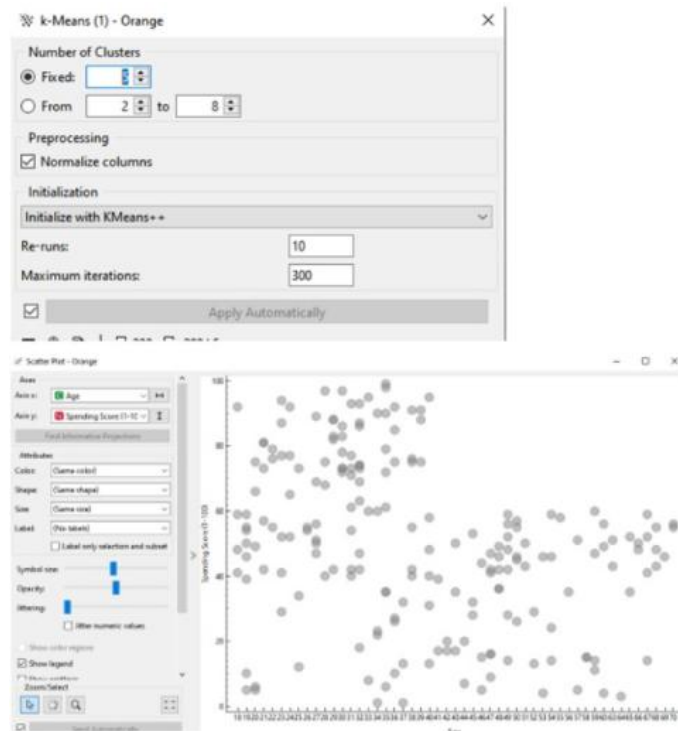
	CustomerID	Genre	Age	annual income (k)	ending Score (1-1)
1	1	Male	19	15.0	39
2	2	Male	21	15.0	81
3	3	Female	20	16.0	6
4	4	Female	23	16.0	77
5	5	Female	31	17.0	40
6	6	Female	22	17.0	76
7	7	Female	35	18.0	6
8	8	Female	23	18.0	94
9	9	Male	64	19.0	3
10	10	Female	30	19.0	72
11	11	Male	67	19.0	14
12	12	Female	35	19.0	99
13	13	Female	58	20.0	15
14	14	Female	24	20.0	77
15	15	Male	37	20.0	13
16	16	Male	22	20.0	79
17	17	Female	35	21.0	35
18	18	Male	20	21.0	66
19	19	Male	52	23.0	29



CONFIGURATION AND VISUALISATION:

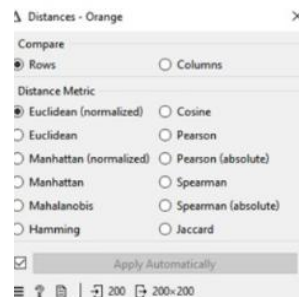
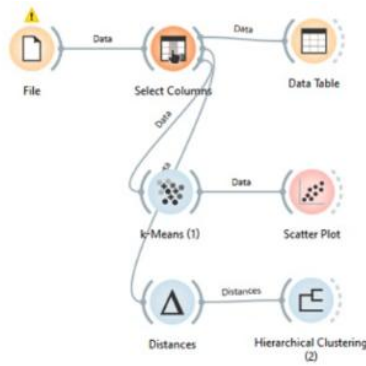
In the configuration stage, the **K-Means** widget is connected to the selected customer dataset, and the number of clusters (**k**) is specified based on the desired segmentation analysis. In this lab, the value of **k = 5** is commonly used to create five distinct customer groups. The algorithm processes the selected features, such as **Annual Income** and **Spending Score**, and assigns each customer to the nearest cluster centroid.

For visualization, the output of the K-Means widget is connected to the **Scatter Plot** widget in Orange Data Mining. The X-axis is assigned to **Annual Income**, while the Y-axis represents **Spending Score**. Different colors are used to display separate clusters, making it easier to visually identify customer groups such as high-income/high-spending customers or low-income/low-spending customers. This graphical representation helps analysts better understand customer behavior patterns and evaluate the effectiveness of the clustering process.



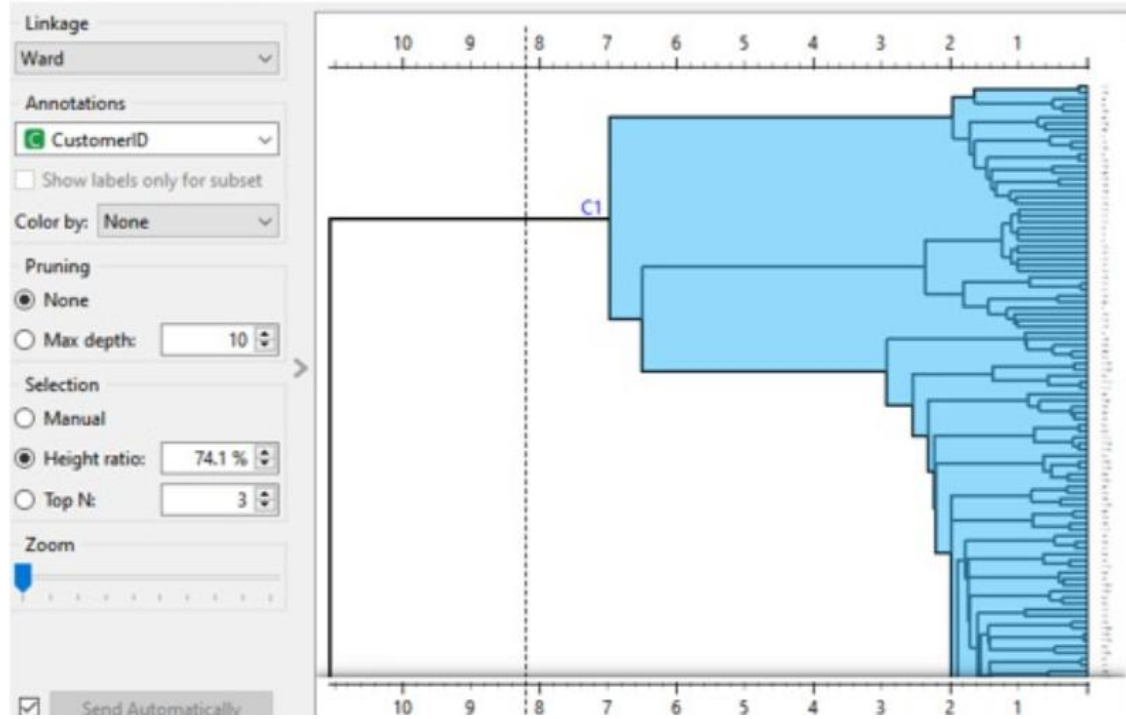
Drag the Hierarchical Clustering widget.

Add the **Hierarchical Clustering** widget to the Orange Data Mining workspace and connect it with the output of the **Distances** widget. This widget is used to create a dendrogram that visually represents the similarity and grouping structure among customers. By analyzing the dendrogram branches, users can identify the most suitable number of customer clusters before applying the K-Means algorithm.



Visualization:

Visualization is used to graphically represent the customer clusters generated by the clustering algorithms. In Orange Data Mining, the **Scatter Plot** widget is connected to the output of the **K-Means** model to display customer groups visually. The X-axis is typically assigned to **Annual Income**, while the Y-axis represents the **Spending Score**. Each cluster is shown using different colors, allowing analysts to clearly observe the separation between customer segments. This visual representation helps identify important groups such as high-income/high-spending customers, low-income/low-spending customers,



Lab No 13: Outlier Detection

1. Introduction

In the banking and financial sector, billions of data points are processed every single day. Most of this activity follows predictable, standardized patterns representing everyday consumer behavior. However, hidden within this massive volume of normal activity are statistical anomalies—data points that deviate drastically from the norm. These outliers can represent anything from simple data entry errors to highly sophisticated fraudulent transactions or abnormal account behaviors. If a bank's security system cannot identify these deviations, they risk exposing their customers and their own bottom line to severe financial threats. Outlier detection serves as a vital first line of defense. By using algorithmic techniques, we can automatically isolate these suspicious data points for further investigation. This laboratory module explores how to identify abnormal banking behaviors using both distance-based heuristics and tree-based Isolation Forest methodologies.

2. Problem Statement

A multinational bank is attempting to secure its retail banking division against emerging financial threats. Currently, their internal audit team relies on manual, rule-based thresholds (e.g., flagging any account with a balance over a rigid dollar amount) to catch suspicious activity. This approach is incredibly inefficient, misses nuanced fraud, and produces a massive amount of false positives. They have extracted a historical dataset of customer financial standings and account activities. The analytics department needs us to design an automated outlier detection pipeline. The challenge is to deploy machine learning algorithms that can detect multivariate anomalies—such as a customer with a suspiciously low salary but an inexplicably high account balance—without relying on outdated manual rules. Implementing this will allow the bank to precisely flag high-risk profiles while keeping false alarms to an absolute minimum.

3. Dataset Description

For this practical exercise, we will utilize the **Banking Dataset** (provided locally as Churn_Modelling.csv). While this structured tabular file is occasionally used for churn prediction, its rich numerical features make it an excellent sandbox for unsupervised anomaly detection. To successfully perform the required data mining tasks, students must familiarize themselves with the following key attributes:

- **Identification Metrics:** RowNumber, CustomerId, and Surname. These serve as structural keys but hold no mathematical value for distance calculations and must be excluded.
- **Financial Indicators:** CreditScore, Balance, and EstimatedSalary. These continuous numerical fields are our primary targets for detecting financial irregularities.
- **Demographic & Engagement Data:** Age, Tenure, and NumOfProducts. These variables help contextualize the financial indicators. For instance, an extremely high balance might be typical for a 65-year-old with a 10-year tenure, but highly anomalous for an 18-year-old new customer.

The screenshot shows the Kaggle interface for the 'Banking Customer Churn Prediction Dataset'. On the left is a sidebar with navigation options like 'Home', 'Competitions', 'Benchmarks', 'Game Arena', 'Data Hub', 'More', 'Your Work', and 'Viewed'. The main content area features a search bar, a user profile for 'SAURABH BADOLE' (updated 2 years ago), and a '190' rating. The dataset title 'Banking Customer Churn Prediction Dataset' is prominently displayed, followed by the subtitle 'Understanding Customer Behavior and Predicting Churn in Banking Institutions'. Below this are tabs for 'Data Card', 'Code (86)', 'Discussion (5)', and 'Suggestions (0)'. The 'About Dataset' section includes a 'Description' and 'Features' (starting with 'RowNumber'). On the right, there are sections for 'Usability' (10.00), 'License' (Attribution-NonCommercial-No...), 'Expected update frequency' (Never), and 'Tags' (Business, Classification, Data Analytics, Beginner). A visual representation of the dataset shows five white stick figures and one yellow stick figure walking away.

4. Tools Used

This procedure relies on the integration of **RapidMiner Studio** alongside embedded **Python** scripting. RapidMiner Studio is an advanced visual analytics platform that allows us to rapidly prototype our data pipeline and apply out-of-the-box distance-based anomaly operators without getting bogged down in syntax. However, to implement the highly efficient Isolation Forest algorithm, we will utilize RapidMiner's native Execute Python operator. This dual-tool framework provides analysts with both the rapid prototyping of a drag-and-drop interface and the algorithmic granularity of Python's machine learning libraries.

5. Practical Objectives

Upon successful completion of this laboratory session, students will demonstrate practical competency by achieving the following learning outcomes:

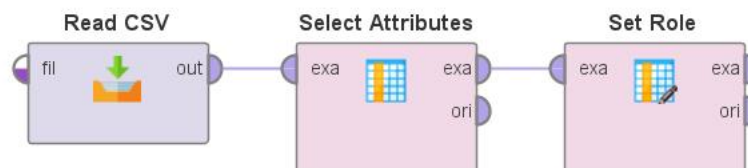
- **Ingest and preprocess** raw banking records within the RapidMiner workspace to remove irrelevant identification features.
- **Deploy** visual operators to execute a distance-based nearest neighbor anomaly detection algorithm.
- **Implement** an embedded Python script to apply the Isolation Forest technique for multidimensional outlier scoring.
- **Evaluate** the operational effectiveness of the detection models by calculating specific KPIs: Anomaly Detection Rate and False Alarm Rate.

6. Step-by-Step Procedure

The following workflow systematically applies anomaly detection principles to the banking dataset.

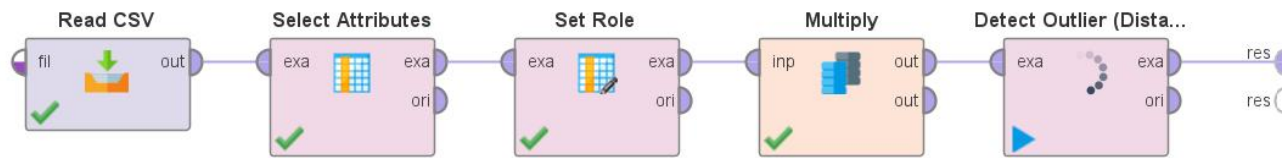
Step 1: Data Ingestion and Feature Selection

- **Action:** Load the raw banking dataset and filter out non-predictive text fields so the mathematical algorithms can process the numerical space.
- **Configuration:** Locate the Read CSV operator in the repository and drop it onto the main process canvas. Double-click the node to import the Churn_Modelling.csv file. Next, attach a Select Attributes operator. In the parameter panel, change the filter type to "subset" and explicitly remove RowNumber, CustomerId, and Surname.
- **Execution:** Route the output port to the results node at the edge of the canvas and execute the process.
- **Verification:** Transition to the Results tab. Confirm that the data grid now strictly contains analytical variables (like Balance, Age, and EstimatedSalary) and is completely free of identification noise.



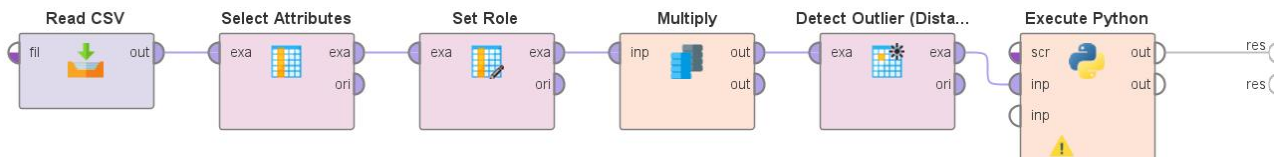
Step 2: Distance-Based Anomaly Detection

- **Action:** Apply an algorithm that flags rows as outliers based on how far away they sit mathematically from their nearest neighbors in the data space.
- **Configuration:** Disconnect the results node and insert a Detect Outlier (Distances) operator directly following your attribute selection step. In the parameters panel, set the number of neighbors (k) to 10 and set the number of outliers to flag to 50.
- **Execution:** Run the pipeline. The operator calculates the Euclidean distance between all customer profiles to find the most isolated points.
- **Verification:** Open the newly generated output table. You will observe a new boolean column appended to the dataset, explicitly labeling exactly 50 customer instances as "true" for being outliers based on their extreme mathematical distance from the average banking customer.



Step 3: Programmatic Isolation Forest Detection

- **Action:** Utilize a tree-based algorithm via Python to isolate anomalies. This method works on the principle that outliers are "few and different," making them much easier to separate into distinct branches than normal data points.
- **Configuration:** Remove the distance-based operator from the canvas and replace it with an Execute Python operator. Map the selected attributes data stream into its primary input port.
- **Settings:** Within the operator's script editor, construct a concise routine. Import IsolationForest from sklearn.ensemble. Initialize the model with a contamination parameter of 0.05 (telling the model to expect a 5% anomaly rate). Run the fit_predict function on your dataset and append the resulting array (-1 for anomaly, 1 for normal) as a new column named Isolation_Score.
- **Execution:** Route the script's output to the results node and run the final pipeline.
- **Verification:** Inspect the final data grid. Sort the table by the new Isolation_Score column to immediately view all the customer rows that the Python script has flagged with a -1, representing our isolated banking anomalies.



7. Key Performance Indicator (KPI) Calculations

To bridge the gap between technical detection algorithms and banking security reporting, analysts must translate the output of their pipeline into standardized performance metrics.

7.1 Anomaly Detection Rate (True Positive Rate)

- **Action:** Measure the exact proportion of genuinely abnormal or fraudulent transactions that the model successfully identified.
- **Calculation Method:** $(\text{True Anomalies Detected} / \text{Total Actual Anomalies in Dataset}) \times 100$
- **Operator Extraction:** If evaluating against a labeled validation set, integrate a Performance (Classification) operator. Locate the True Positive count in the generated confusion matrix and divide it by the total number of known anomalies. This dictates the core sensitivity of the bank's new detection system.

7.2 False Alarm Rate (False Positive Rate)

- **Action:** Determine how often the system incorrectly flags a perfectly normal customer account as a suspicious outlier, a critical metric since false alarms waste the audit team's time.
- **Calculation Method:** $(\text{Normal Accounts Flagged as Outliers} / \text{Total Normal Accounts}) \times 100$
- **Operator Extraction:** Utilizing the same Performance operator's confusion matrix, divide the False Positive count by the total actual negative (normal) instances. A lower percentage here proves that the model is highly precise and operationally efficient.

8. KPI Summary Table

Compile your findings into the following matrix to provide a concise technical reference for the model's evaluation phase.

KPI Name	Calculation Method	RapidMiner / Python Modules Utilized
Anomaly Detection Rate	$(\text{True Positives} / \text{Total Actual Anomalies}) \times 100$	Execute Python (Isolation Forest), Performance Matrix
False Alarm Rate	$(\text{False Positives} / \text{Total Actual Normal Records}) \times 100$	Detect Outlier (Distances), Performance Matrix

9. Conclusion

This finalizes the laboratory module on unsupervised machine learning and outlier detection. We successfully engineered a data pipeline that ingested raw banking ledgers, discarded non-predictive demographic noise, and utilized distance-based heuristics to map out normal customer behavior. By switching to an embedded Python script, we also deployed the highly efficient Isolation Forest algorithm to partition and flag multidimensional anomalies. Armed with these specific risk profiles and the ability to track our Anomaly Detection and False Alarm Rates, the bank's security and audit teams can now transition from outdated, manual threshold rules to a dynamic, machine-learning-driven approach for identifying abnormal financial transactions.

Lab 14: Scraping & Sentiment Analysis – Product Review Analysis

Introduction: In today's digital world, customers mostly rely on online product reviews before making a purchase decision. These reviews contain valuable opinions, experiences, and feedback that can help businesses understand customer satisfaction and improve their products. However, this information is usually unstructured and available on different websites.

Web scraping is the process of automatically collecting data from websites, such as product reviews, ratings, and comments. After collecting this data, sentiment analysis is applied to understand the emotional tone behind the text, whether it is positive, negative, or neutral.

By combining scraping and sentiment analysis, businesses can efficiently analyze large amounts of customer feedback, identify trends, detect problems, and make data-driven decisions to improve product quality and customer experience.

Objective

To extract (scrape) product reviews from social media or review platforms, clean the unstructured text, and perform sentiment classification to determine polarity (positive, negative, neutral).

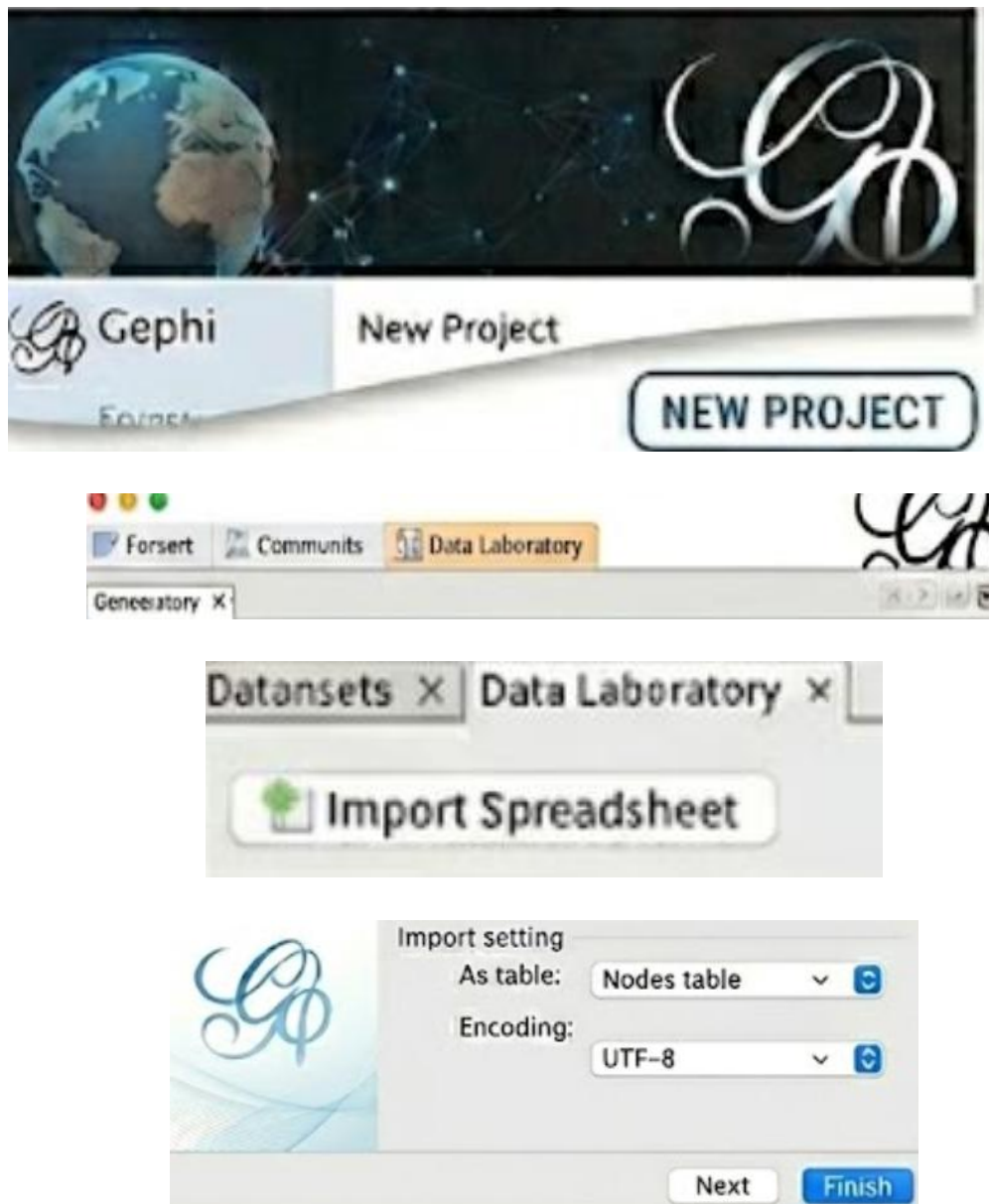
Tool & Dataset

- **Tool:** CiteSpace / Gephi + Python.
- **Dataset:** Social Media / Review Dataset.

Workflow Steps

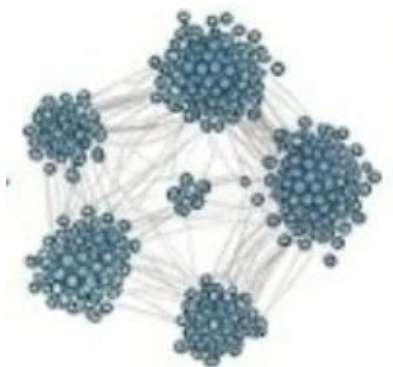
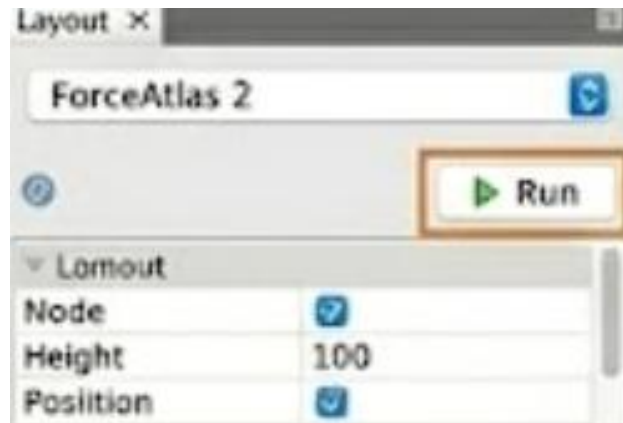
Step 1 – Data Import

- Import the dataset (e.g., scraped reviews) into Gephi using **Import Spreadsheet**.
- Data is read as a **Nodes Table** (entities such as words or users) and **Edges Table** (connections between them).
- Encoding is set to **UTF-8** to ensure proper handling of text data.



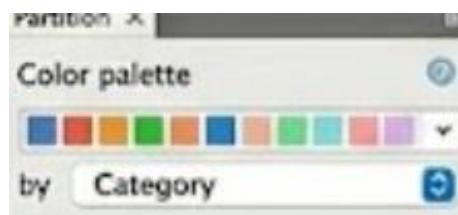
Step 2 – Network Layout

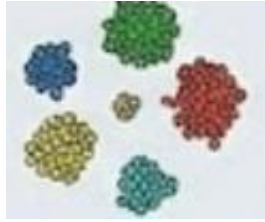
- Apply the **ForceAtlas 2** layout algorithm to arrange nodes visually.
- This algorithm simulates physical forces: connected nodes attract each other, while unrelated nodes repel.
- The result is a clear cluster structure showing groups of related words or reviewers.



Step 3 - Color & Categorization

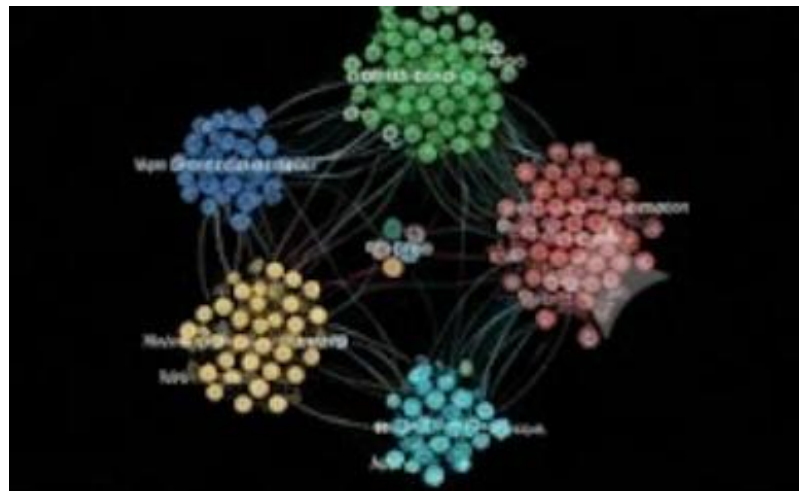
- Use the **Color Palette** option to assign colors by category (e.g., sentiment polarity or topic).
- Each color represents a distinct sentiment cluster — for example, blue for positive, red for negative, and yellow for neutral.





Step 4 – Visualization & Export

- After running the layout, visualize the network graph.
- Export the final visualization using the **Export** window (filename, format, etc.).
- The output shows interconnected clusters representing how sentiments or topics relate across reviews.



PYTHON IMPLEMENTATION :

1. Load and Clean Data

```
import pandas as pd
import re

# Load data with encoding fix
df = pd.read_csv("test (1).csv", encoding='latin-1')
text_col = 'text' # Update if your column name is different

# Simple cleaning function
def clean(text):
    text = re.sub(r'[^a-zA-Z\s]', '', str(text).lower())
    return [w for w in text.split() if w not in ['the', 'and', 'a', 'to', 'of']]

df['clean_tokens'] = df[text_col].apply(clean)
```

2. Sentiment Scoring Engine

```
, 'great', 'love', 'amazing', 'best', 'nice', 'happy']
, 'worst', 'hate', 'terrible', 'poor', 'slow', 'error']

ity(tokens):
    for w in tokens if w in pos)
    for w in tokens if w in neg)
    - n) / (p + n) if (p + n) > 0 else 0

df['clean_tokens'].apply(get_polarity)
'] = df['score'].apply(lambda s: 'Positive' if s > 0 else ('Negative' if s < 0 else 'Neutral'))
4
```

3. KPI Metrics Calculation ¶

```
# KPI 1: Distribution
dist = df['sentiment'].value_counts(normalize=True) * 100
print("--- Sentiment Distribution (%) ---")
print(dist)

# KPI 2: Polarity Summary
print("\n--- Polarity Statistics ---")
print(f"Avg Polarity: {df['score'].mean():.4f}")
print(f"Max Polarity: {df['score'].max():.4f}")
print(f"Min Polarity: {df['score'].min():.4f}")
```

--- Sentiment Distribution (%) ---

sentiment

Neutral 85.503634

Positive 12.107996

Negative 2.388370

Name: proportion, dtype: float64

--- Polarity Statistics ---

Avg Polarity: 0.0973

Max Polarity: 1.0000

Min Polarity: -1.0000

4. Visualization

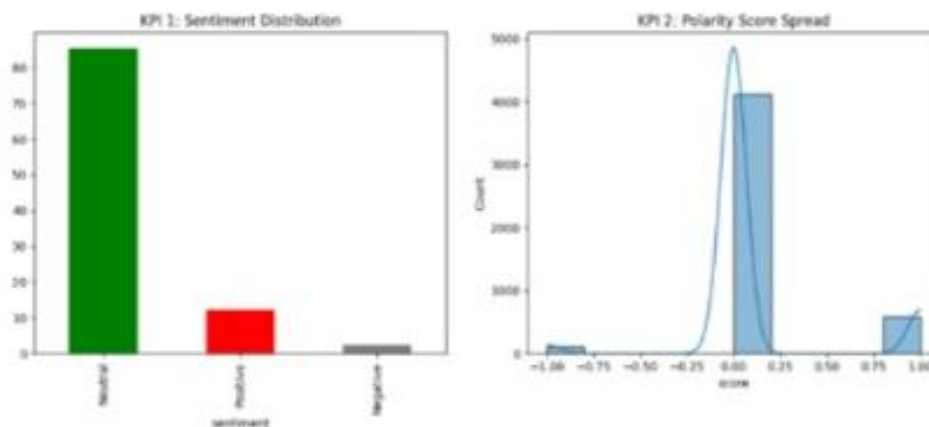
```
import matplotlib.pyplot as plt
import seaborn as sns

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))

# Plot Distribution
dist.plot(kind='bar', ax=ax1, color=['green', 'red', 'gray'])
ax1.set_title("KPI 1: Sentiment Distribution")

# Plot Polarity Spread
sns.histplot(df['score'], ax=ax2, bins=10, kde=True)
ax2.set_title("KPI 2: Polarity Score Spread")

plt.show()
```



Lab 15: Final Capstone – End-to-End Business Solution

This final capstone lab focuses on building a complete end-to-end business solution by integrating all the concepts learned throughout the course. It involves understanding a real-world problem, collecting and processing relevant data, applying analytical or machine learning techniques, and presenting actionable insights.

In this lab, students are expected to design a fully functional workflow that may include data collection, preprocessing, model development, evaluation, and visualization. The goal is to simulate a real business environment where data-driven decision-making is essential.

By completing this capstone project, students demonstrate their ability to apply theoretical knowledge to practical scenarios, solve complex problems, and deliver a professional-level analytical solution that can support business decision-making.

Objective

To design, implement, and document a complete data science pipeline—from data acquisition and preprocessing to modeling, comparison, and business recommendation—using a real-world dataset of your choice.

1. Data Preprocessing

Before training any machine-learning model, raw data must be cleaned and transformed into a usable format.

- **Raw Features:** These are the original attributes collected from datasets (e.g., numeric, categorical, or text values). They often contain inconsistencies, missing values, or varying scales.
- **Processed Data:** After cleaning, the dataset becomes structured and ready for analysis. Each record is standardized, and irrelevant or noisy features are removed.
- **Feature Scaling:** Ensures that all attributes contribute equally to the model's learning process. Without scaling, features with larger numeric ranges can dominate the training process.

Before					After	
	LMM_values	uKIM_values	abdic_sabes	coody_sabes	After	Status
ramon 1	1600	1600	1400	1600	5 400	Picessed
ramon 2	1600	1600	1400	1600	5 600	Picessed
ramon 3	1500	1000	1500	1600	5 600	Picessed
ramon 4	1500	1170	1500	1600	5 600	Picessed
ramon 5	1600	1600	1500	1600	5 600	Picessed

MODELS			8 RAW FEATURES			
Models	Status					
1 DECISION TREE	0.35					
2 NAIVE BAYES	0.34					
3 MKN	0.35					
4 MYN	8.33					
5 SVM	0.39					
6 LOGISTIC DEGRESSION	0.95					
7 RANDOM FOREST	0.82					
8 GRADIENT FOREST	0.36					
9 GRADIENT BOOSTING	0.32					
10 NEURAL NETWORK	0.92					

H SCALED FEATURES			
	Regisit	Seamis	Scaled
Labo 2	0.3587	-0.000	6.000
Labo 4	9.3587	0.000	0.000
Labo d	0.4050	0.000	0.000
Utko Delas	0.4200	4.000	0.000
Labo 3	1.1988	3.900	0.000
Labo 11	0.3991	2.000	0.078

2. Z- Score Normalization

Z- Score normalization (also called standardization) transforms data so that each feature has a mean = 0 and standard deviation = 1. The formula is:

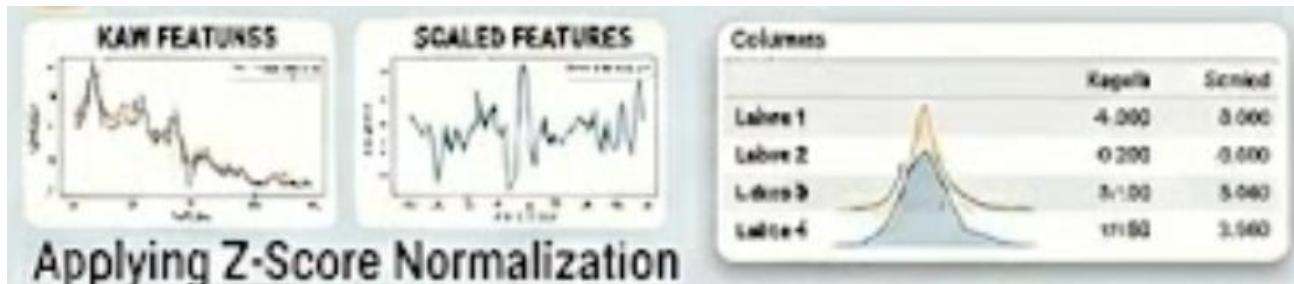
$$Z = \frac{X - \mu}{\sigma}$$

where X is the original value, μ is the mean, and σ is the standard deviation.

Purpose:

- Removes bias caused by differing scales.
- Improves convergence speed for algorithms like SVM, KNN, and Neural Networks.
- Makes data comparable across features.

Effect: After applying Z- Score normalization, the dataset changes from “Before” (raw numeric values) to “After” (scaled values). Graphs of raw vs. scaled features show smoother distributions centered around 0.



3. Model Training

Once data is normalized, multiple algorithms are trained to find the best predictive model. Common models include:

- **Decision Tree:** Splits data based on attribute values to form hierarchical decision rules.
- **Naive Bayes:** Probabilistic classifier based on Bayes' theorem; assumes feature independence.
- **K- Nearest Neighbor (KNN):** Classifies data based on proximity to neighboring points.
- **Support Vector Machine (SVM):** Finds the optimal hyperplane separating classes.
- **Logistic Regression:** Models binary outcomes using a logistic function.
- **Random Forest:** Ensemble of decision trees that improves accuracy and reduces overfitting.
- **Gradient Boosting:** Sequentially builds models to correct previous errors.
- **Neural Network:** Mimics human brain structure using interconnected layers for complex pattern learning.



4. Modeling Pipeline

The **modeling pipeline** automates the workflow: **Clean Data** → **Feature Scaling** → **Model Training** → **Evaluation** → **Comparison**. Each model block receives the same processed dataset, ensuring fair comparison.



5. Model Evaluation

Performance is measured using key metrics:

Metric	Description
--------	-------------

Accuracy	Percentage of correctly predicted instances.
-----------------	--

Metric	Description
--------	-------------

Precision	Ratio of true positives to all predicted positives.
------------------	---

Recall	Ratio of true positives to all actual positives.
---------------	--

F1-Score	Harmonic mean of precision and recall.
-----------------	--

ROC-AUC Measures the model’s ability to distinguish between classes.

The **Model Comparison Table** ranks algorithms based on these metrics.

- Models like **Naive Bayes**, **KNN**, and **Gradient Boosting** often achieve high ROC-AUC scores.
- The **Best Model** is selected based on balanced performance across all metrics.

	Accuracy	Precision	Recall	F1-Score	ROC-AUC
(1) Decision Tree	0.99	0.92	0.99	0.99	0.99
(2) Naive Bayes	0.87	0.86	0.87	0.76	0.99
(2) KNN	0.96	0.83	0.83	0.56	0.96
(4) SVM	0.53	0.58	0.52	0.76	0.49
(5) Logistic Regression	0.95	0.95	0.95	0.58	0.96
(6) Random Forest	0.94	0.96	0.83	0.57	0.96
(7) Gradient Boosting	0.93	0.96	0.55	0.56	0.37
(8) Neural Network	0.83	0.53	0.34	0.55	0.39

6. Outcome

After evaluation, the workflow identifies the most accurate and reliable model for prediction. The **ROC-AUC Comparison** and **Best Model Graph** visualize performance differences, helping analysts choose the optimal algorithm for deployment.

